
**Proyecto
GROW**



Aplicación de gestión de gastos personales Grow
Plan de Desarrollo de Software y Documentación Técnica

Versión 1.0

Hoja de control del documento

Título	Plan de desarrollo de software y documentación técnica. Aplicación de gestión de gastos personales Grow.		
Descripción	Grow es una aplicación de escritorio desarrollada para facilitar el registro, almacenamiento, consulta, filtrado, análisis y exportación de gastos personales. El sistema permite organizar los gastos por categoría, visualizar estadísticas y generar reportes en formato PDF y Excel.		
Autor	Aurora Mendoza Borunda		
Responsable	Aurora Mendoza Borunda		
Organización	Universidad Autónoma de Chihuahua. Facultad de Ingeniería. Licenciatura en Ingeniería de Software.		
Categoría	Documentación técnica		
Palabras clave	gastos personales, Flutter Desktop, Dart, SQLite, software, reportes PDF, reportes Excel, pruebas de software, GitHub Pages		
Versión	1.0	Fecha	05 de agosto de 2026
Edición	1 revisión		

Historial de revisiones

Fecha	Versión	Descripción	Autor
2026-08-05	1.0	Versión inicial	Aurora Mendoza Borunda

Tabla de contenidos

1 Introducción	7
1.1 Propósito.....	7
1.2 Usuarios.....	8
1.3 Alcance	9
1.4 Resumen.....	10
1.5 Advertencias	11
1.6 Glosario, acrónimos y definiciones.....	11
2 Vista general del proyecto	14
2.1 Propósito, alcance y objetivos.....	14
2.2 Restricciones y suposiciones	15
2.2.1 Restricciones.....	16
2.2.2 Suposiciones	16
2.3 Ciclo de vida.....	17
2.3.1 Especificación.....	17
2.3.2 Diseño y arquitectura	17
2.3.3 Programación	17
2.3.4 Prueba.....	17
2.3.5 Mantenimiento y mejora.....	18
2.4 Metodología	20
2.5 Entregables del proyecto	21
3 Organización del proyecto.....	23
3.1 Recursos humanos.....	23
3.2 Roles y responsabilidades	24
4 Gestión de procesos del proyecto	29
4.1 Plan de actividades	29
4.2 Seguimiento y control.....	31
4.3 Gestión de configuración	34
5 Infraestructura.....	37
5.1 Hardware	37
5.2 Software.....	38
5.3 Herramientas utilizadas	39
6 Diseño de la arquitectura del sistema	41
6.1 Descripción general.....	41
6.2 Estructura del proyecto.....	43
6.3 Base de datos	44
6.4 Componentes principales	45
7 Ingeniería de requerimientos	47
7.1 Requerimientos funcionales.....	48
7.2 Requerimientos no funcionales.....	49
7.3 Restricciones.....	50
8 Procesos del sistema.....	51
8.1 Actores.....	51

8.2 Casos de uso	52
8.3 Procesos principales	55
9 Interfaces del sistema.....	59
9.1 Pantalla principal	59
9.2 Pantalla de registro de gastos	60
9.3 Pantalla de estadísticas.....	61
9.4 Exportación de reportes.....	62
10 Componentes de software.....	64
10.1 Modelo Expense	64
10.2 DatabaseHelper.....	65
10.3 StatisticsService	66
10.4 Servicios de exportación PDF y Excel.....	67
11 Gestión de riesgos.....	70
11.1 Riesgos técnicos	70
11.2 Riesgos de datos.....	71
11.3 Árboles de fallos	73
12 Verificación de calidad	76
12.1 Pruebas unitarias.....	76
12.2 Pruebas de widgets	77
12.3 Pruebas de rendimiento.....	77
12.4 Resultados obtenidos	78
13 Publicación y control de versiones.....	79
13.1 Git	80
13.2 GitHub.....	81
13.3 GitHub Pages	82
14 Manual básico de uso.....	85
14.1 Git	85
14.2 Git	85
14.3 Git	86
14.4 Git	87
14.5 Git	88
14.6 Git	89
14.7 Git	90
14.8 Git	91
14.9 Git	93
14.10 Git	94
14.11 Git	95
15 Estado actual del proyecto y conclusiones personales	96
16 Anexos gráficos	99
16.1 Diagrama UML de clases.....	99
16.2 Diagrama ER.....	99
16.3 Interfaces	100
16.4 Pantallas de pruebas de software con Flutter Test.....	107
16.5 Pantallas de rendimiento con Flutter DevTools.....	109
16.6 Árboles de fallos con ALD.....	110

17 Bibliografía.....	113
-----------------------------	------------

Tabla de ilustraciones

Ilustración 1. Interacciones entre actores en Grow	26
Ilustración 2. Diagrama de Gantt de Grow	31
Ilustración 3. Estructura de Grow.....	34,43
Ilustración 4. Diagrama de arquitectura lógica del sistema Grow	42
Ilustración 5. Logo de Grow	85
Ilustración 6. Pantalla principal de Grow.....	86,100
Ilustración 7. Pantalla de registro de gastos.....	87,100
Ilustración 8. Pantalla de lista de gastos registrados.....	88,101
Ilustración 9. Pantalla de filtro de gastos por categoría.....	89,101
Ilustración 10. Pantalla de lista de gastos filtrados por categoría “Transporte”.....	89,102
Ilustración 11. Pantalla de botón de eliminación de gasto.....	90,102
Ilustración 12. Pantalla de gráfico de gastos.....	91,103
Ilustración 13. Pantalla de botón “Exportar PDF”	91,103
Ilustración 14. Pantalla de mensaje de exportación de PDF.....	92,104
Ilustración 15. Pantalla del archivo PDF exportado.....	92,104
Ilustración 16. Pantalla de botón de exportación de Excel.....	93,105
Ilustración 17. Pantalla de mensaje de exportación de Excel.....	93,105
Ilustración 18. Pantalla de archivo Excel exportado.....	94,106
Ilustración 19. Pantalla principal del sitio web de Grow.....	95,106
Ilustración 20. Diagrama UML de clases.....	114
Ilustración 21. Diagrama Entidad-Relación de Grow	114
Ilustración 22. Evidencia de pruebas unitarias exitosas	107
Ilustración 23. Evidencia de pruebas de widgets exitosas	108
Ilustración 24. Evidencia de prueba de rendimiento aceptable	110
Ilustración 25. Evidencia 1 de creación de árboles de fallos en ALD.....	111
Ilustración 26. Evidencia 2 de árbol de fallos en ALD	111
Ilustración 27. Evidencia 3 de creación de árboles de fallos en ALD.....	112

1 Introducción

El presente documento técnico tiene como finalidad describir el desarrollo, funcionamiento y características principales de la aplicación de escritorio denominada a partir de ahora **Grow**, un sistema orientado a la gestión de gastos personales. La aplicación fue desarrollada con Flutter Desktop y utiliza una base de datos local SQLite para almacenar la información ingresada por el usuario.

Grow permite registrar gastos personales mediante datos como monto, categoría, descripción y fecha. Además, el sistema permite consultar los registros guardados, filtrar gastos por categoría, visualizar estadísticas generales y generar reportes en formatos PDF y Excel. Así, este documento proporciona una visión general del proyecto, su propósito, sus usuarios, su alcance, sus restricciones, su estructura técnica y los principales conceptos usados durante su desarrollo. Asimismo, sirve como referencia para comprender las decisiones de diseño, las herramientas empleadas, los componentes implementados y los mecanismos de validación aplicados al sistema.

La documentación técnica de Grow se elaboró como parte del proceso de desarrollo del proyecto, con el propósito de registrar de manera ordenada los aspectos esenciales de la aplicación y facilitar su revisión, mantenimiento y mejora futura.

1.1 Propósito

El propósito de este documento, a partir de ahora referido con el acrónimo PDSDT (Plan de Desarrollo del Software y Documento Técnico), es proporcionar la información necesaria para comprender, controlar y documentar el proyecto Grow, así como describir su funcionalidad y las soluciones técnicas adoptadas durante su construcción.

De manera específica, este documento tiene los siguientes propósitos:

- Documentar de forma clara los elementos principales que conforman el proyecto Grow.
- Describir la funcionalidad general de la aplicación.
- Presentar las herramientas y tecnologías utilizadas durante el desarrollo.
- Explicar la estructura del proyecto y sus principales componentes de software.
- Registrar las funcionalidades implementadas en la aplicación.
- Describir las pruebas realizadas para verificar el funcionamiento del sistema.
- Documentar los mecanismos de validación, rendimiento y análisis de fallos aplicados al proyecto.
- Servir como base para futuras modificaciones, mejoras o mantenimiento

El documento también busca evidenciar el proceso de desarrollo seguido en Grow desde la planeación inicial hasta la implementación, prueba, documentación y publicación del sitio web del proyecto.

12 Usuarios

El documento está dirigido principalmente a personas interesadas en conocer el desarrollo técnico de la aplicación Grow. Por tanto, a diferencia de un manual exclusivamente orientado al usuario final, tiene un enfoque técnico y académico de modo que los usuarios previstos de esta documentación son los siguientes:

- *Desarrolladora del proyecto*: hace uso del documento como guía de referencia para explicar el funcionamiento, estructura, herramientas y decisiones técnicas empleadas en Grow.
- *Docente o evaluadora académica*: consulta el documento para verificar el cumplimiento de los objetivos del proyecto, las funcionalidades implementadas, las pruebas realizadas y la documentación generada.

- *Usuarios finales:* pueden consultar algunas secciones del documento, especialmente la descripción general, funcionalidades principales y manual básico de uso, para comprender cómo utilizar la aplicación.
- *Futuros desarrolladores o responsables de mantenimiento:* pueden usar esta documentación para identificar la estructura del proyecto, los componentes principales, la base de datos, las herramientas usadas y las posibles áreas de mejora.

13 Alcance

El alcance de este documento comprende la descripción técnica y funcional de la aplicación de escritorio **Grow** desarrollada para la gestión básica de gastos personales. Aquí se abarcan los siguientes aspectos:

- Descripción general del proyecto.
- Objetivo del sistema.
- Usuarios previstos.
- Funcionalidades principales.
- Herramientas y tecnologías utilizadas.
- Estructura del proyecto.
- Diseño general de la aplicación.
- Uso de base de datos local SQLite.
- Registro, consulta, filtrado y eliminación de gastos.
- Generación de estadísticas.
- Exportación de reportes PDF y Excel.
- Pruebas unitarias y pruebas de widgets con Flutter Test.
- Evaluación de rendimiento mediante Flutter DevTools.
- Análisis cualitativo de fallos mediante árboles de fallos elaborados con ALD.
- Uso de Git y GitHub para control de versiones.
- Publicación de documentación mediante GitHub Pages.

El alcance del sistema se limita a una aplicación de escritorio para Windows. Grow no contempla, en esta versión, sincronización en la nube, inicio de sesión de usuarios, conexión con cuentas bancarias, respaldo automático en línea ni uso desde dispositivos móviles. Por lo tanto, la aplicación se enfoca en ofrecer una solución local, sencilla y funcional para registrar y analizar gastos personales desde un equipo de escritorio.

14 Resumen

Este PDSDT se organiza en diferentes apartados que permiten comprender el proyecto **Grow** desde una perspectiva general, funcional y técnica. En primer lugar, se presenta la introducción del proyecto, su propósito, los usuarios previstos y el alcance de la documentación. Posteriormente, se describe la vista general del sistema, incluyendo los objetivos principales, restricciones, suposiciones y metodología de desarrollo. Después, se documentan las herramientas utilizadas durante el proyecto, entre ellas Flutter Desktop, Dart, SQLite, Visual Studio Code, Git, GitHub, GitHub Pages, Flutter Test, Flutter DevTools y ALD Free Web Fault Tree Analysis Tool. Asimismo, se describen las funcionalidades principales del sistema, como el registro de gastos, almacenamiento local, visualización de gastos, filtrado por categoría, estadísticas y exportación de reportes en PDF y Excel. Finalmente, el documento incluye secciones dedicadas a la verificación de calidad, pruebas realizadas, análisis de rendimiento, análisis de árboles de fallos, control de versiones, publicación web, estado actual del proyecto, conclusión y anexos con evidencias del desarrollo.

15 Advertencias

El presente PDSDT describe el estado actual del proyecto **Grow** con base en las funcionalidades desarrolladas hasta el momento. Por lo tanto, algunos apartados podrían modificarse en futuras versiones si se agregan nuevas características, se cambia la estructura del proyecto o se incorporan nuevas herramientas.

La aplicación Grow fue desarrollada con fines académicos por lo cual su alcance está limitado a la gestión básica de gastos personales en un entorno local de escritorio; aunque el sistema permite registrar, consultar, analizar y exportar información financiera, no debe considerarse una herramienta financiera profesional ni sustituye sistemas contables especializados.

Análogamente, la información registrada en Grow se almacena localmente mediante SQLite lo cual significa que los datos dependen del equipo donde se ejecuta la aplicación y no se sincronizan automáticamente con servicios externos o almacenamiento en la nube. Así, las pruebas descritas en este documento corresponden a escenarios definidos durante el desarrollo académico del proyecto. Aunque permiten validar el funcionamiento general del sistema, no representan una certificación profesional de calidad, seguridad o rendimiento.

Finalmente, los árboles de fallos incluidos en el proyecto se realizaron como análisis cualitativo; no se efectuaron cálculos cuantitativos de probabilidad o tasa de falla, debido a que no se cuenta con datos históricos reales sobre la frecuencia de ocurrencia de cada posible fallo.

16 Glosario, acrónimos y definiciones

Con el

fin de clarificar la terminología utilizada en el presente documento, se adjunta un breve prontuario de acrónimos, siglas y términos de uso específico.

ALD	Herramienta utilizada para elaborar árboles de fallos y analizar posibles eventos críticos dentro del sistema
Arbol de fallos	Técnica de análisis que permite identificar las posibles causas de un evento no deseado dentro de un sistema
Base de datos	Conjunto organizado de información que puede almacenarse, consultarse y modificarse
CRUD	Acrónimo de <i>Create</i> , <i>Read</i> , <i>Update</i> y <i>Delete</i> . Se refiere a las operaciones básicas de crear, leer, actualizar y eliminar información
Dart	Lenguaje de programación utilizado para desarrollar la aplicación Grow
Flutter	Framework de desarrollo utilizado para crear la interfaz y la lógica principal de la aplicación
CRUD	<i>Create</i> , <i>Read</i> , <i>Update</i> , <i>Delete</i> . Acrónimo del proceso de gestión de registros en una base de datos
Flutter Desktop	Versión de Flutter que permite desarrollar aplicaciones de escritorio para sistemas como Windows
Flutter DevTools	Herramienta utilizada para analizar el rendimiento y comportamiento de una aplicación Flutter
Flutter Test	Herramienta utilizada para realizar pruebas unitarias y pruebas de widgets en aplicaciones Flutter

Git	Sistema de control de versiones utilizado para registrar los cambios realizados en el proyecto
GitHub	Plataforma utilizada para almacenar el repositorio remoto del proyecto Grow
GitHub Pages	Servicio utilizado para publicar el sitio web de documentación del proyecto
Grow	Aplicación de escritorio desarrollada para registrar, consultar, filtrar, analizar y exportar gastos personales
Interfaz de usuario	Parte visual de la aplicación mediante la cual el usuario interactúa con el sistema
PDF	Formato de archivo utilizado para generar reportes exportables de gastos
Reporte	documento generado por la aplicación que contiene información organizada sobre los gastos registrados
SQLite	Sistema de base de datos local utilizado para almacenar los gastos registrados en Grow
Software	Conjunto de programas, datos y documentación relacionados con la operación de un sistema informático

VS Code	Visual Studio Code, editor de código utilizado durante el desarrollo del proyecto
Widget	Componente visual utilizado en Flutter para construir la interfaz de usuario

2 Vista general del proyecto

La aplicación **Grow** es un proyecto de desarrollo de software orientado a la gestión de gastos personales mediante una aplicación de escritorio para Windows. El sistema fue desarrollado con Flutter Desktop y utiliza una base de datos local SQLite para almacenar los gastos registrados por el usuario.

Grow permite realizar operaciones básicas de administración financiera personal, tales como registrar gastos, consultar la lista de gastos recientes, filtrar información por categoría, visualizar estadísticas y exportar reportes en formatos PDF y Excel. Su finalidad es ofrecer una herramienta sencilla, funcional y accesible para que el usuario pueda llevar un control básico de sus gastos desde un entorno local.

El proyecto fue desarrollado con fines académicos y contempla tanto la construcción de la aplicación como su documentación técnica, control de versiones, pruebas funcionales, pruebas de rendimiento y análisis preventivo de fallos. Por ello, Grow no solo se considera una aplicación de gestión de gastos, sino también un proyecto integral de ingeniería de software que permite evidenciar las fases de análisis, diseño, implementación, prueba, validación y documentación.

2.1 Propósito, alcance y objetivos

El propósito principal del proyecto **Grow** es desarrollar una aplicación de escritorio que permita al usuario registrar y administrar sus gastos personales de manera sencilla; dicha aplicación busca facilitar el control básico de las finanzas personales mediante el almacenamiento local de información, la consulta de registros, el filtrado por categorías y la generación de reportes.

El alcance del proyecto comprende el desarrollo de una aplicación funcional para escritorio en Windows, construida con Flutter Desktop y Dart. La información se almacena de manera local mediante SQLite por lo cual el sistema no depende de conexión a Internet para registrar o consultar gastos. Además, se integran funciones complementarias como estadísticas, exportación de reportes y documentación del proyecto mediante GitHub Pages.

En términos generales, los objetivos (como fueron mencionados anteriormente) del proyecto son los siguientes:

- Desarrollar una aplicación de escritorio para la gestión de gastos personales.
- Permitir el registro de gastos con monto, categoría, descripción y fecha.
- Almacenar los registros en una base de datos local SQLite.
- Mostrar en pantalla los gastos registrados por el usuario.
- Permitir el filtrado de gastos por categoría.
- Calcular información estadística sobre los gastos registrados.
- Exportar reportes en formato PDF y Excel.
- Aplicar pruebas unitarias y pruebas de widgets mediante Flutter Test.
- Evaluar el rendimiento de la aplicación mediante Flutter DevTools.

- Analizar posibles eventos críticos mediante árboles de fallos.
- Utilizar Git y GitHub para el control de versiones.
- Publicar una página web de documentación mediante GitHub Pages.

22 Restricciones y suposiciones

2.2.1 Restricciones

El desarrollo de Grow está condicionado por diferentes restricciones técnicas, académicas y funcionales. En primer lugar, el proyecto se desarrolla dentro de un contexto académico de modo que sus funcionalidades están delimitadas por los objetivos de aprendizaje, el tiempo disponible y los recursos técnicos usados durante el proceso. Entre las principales restricciones del proyecto se encuentran las siguientes:

- La aplicación está orientada a ejecutarse como aplicación de escritorio en Windows.
- El almacenamiento de información se realiza de forma local mediante SQLite.
- El sistema no cuenta con sincronización en la nube.
- El proyecto no incluye autenticación de usuarios ni manejo de múltiples cuentas.
- La aplicación no se conecta con bancos, tarjetas o servicios financieros externos.
- Las pruebas realizadas corresponden a escenarios académicos definidos durante el desarrollo.
- La exportación de reportes depende del correcto funcionamiento de las librerías utilizadas para PDF y Excel.
- El funcionamiento de la aplicación depende de que Flutter Desktop, Dart y las dependencias del proyecto estén correctamente configuradas.
- El análisis de árbol de fallos se realiza de forma cualitativa, sin cálculos probabilísticos, debido a que no se cuenta con datos históricos reales sobre fallas.

2.2.2 Suposiciones

Para el desarrollo y uso de Grow se consideran las siguientes suposiciones:

- El usuario cuenta con un equipo de escritorio o laptop con sistema operativo Windows.
- El usuario necesita registrar gastos personales de forma sencilla y local.
- El usuario no requiere sincronización en línea para esta versión del sistema.
- Los gastos registrados corresponden a información básica: monto, categoría, descripción y fecha.
- El usuario podrá consultar los registros desde la misma computadora donde se ejecuta la aplicación.
- El ambiente de desarrollo cuenta con Flutter, Dart, Visual Studio Code y las dependencias necesarias.

- La base de datos local SQLite será suficiente para almacenar la información generada por el usuario.
- Las funciones de exportación PDF y Excel serán utilizadas como apoyo para consultar o guardar reportes.
- La documentación técnica y el sitio web del proyecto servirán como evidencia del proceso de desarrollo.

2.3 Ciclo de vida

El ciclo de vida del proyecto Grow se desarrolló mediante un enfoque incremental y basado en prototipos. Este enfoque permitió construir la aplicación por etapas, agregando funcionalidades de manera progresiva y realizando ajustes conforme se identificaban nuevas necesidades o mejoras.

El desarrollo no siguió un modelo rígido en cascada, sino un proceso gradual en el que cada avance permitió obtener una versión funcional del sistema. A partir de esa versión se realizaron modificaciones, correcciones y ampliaciones hasta completar las funciones principales. El uso de un ciclo de vida incremental fue adecuado para Grow porque permitió avanzar paso a paso, verificar cada funcionalidad y realizar mejoras conforme el proyecto evolucionaba.

El ciclo de vida aplicado a Grow puede organizarse en las siguientes fases:

2.3.1 Especificación

En esta fase se definieron las necesidades principales del sistema. Se estableció que Grow debía permitir registrar gastos personales, almacenarlos localmente, consultarlos desde una pantalla principal, filtrarlos por categoría, visualizar estadísticas y exportar reportes.

2.3.2 Diseño y arquitectura

En esta fase se definió la estructura general de la aplicación. Se organizaron carpetas y componentes como modelos, pantallas, servicios, base de datos y widgets. También se eligieron las herramientas principales del proyecto: Flutter Desktop, Dart y SQLite.

2.3.3 Programación

Durante esta fase se implementaron las funciones principales de la aplicación. Se desarrollaron las pantallas de registro de gastos, pantalla principal, estadísticas, conexión con SQLite y servicios de exportación PDF y Excel.

2.3.4 Prueba

En esta fase se verificó el funcionamiento del sistema mediante pruebas unitarias, pruebas de widgets y pruebas de rendimiento. Se utilizaron Flutter

Test y Flutter DevTools para comprobar el comportamiento general de la aplicación.

2.3.5 Mantenimiento y mejora

Finalmente, se realizaron correcciones, ajustes visuales, mejoras en la interfaz, incorporación del logo, publicación del sitio web y documentación del proyecto. Esta fase también contempla posibles mejoras futuras, como agregar nuevas estadísticas, respaldo en la nube o versión móvil.

24 Metodología

La metodología utilizada en **Grow** se basó en un **enfoque ágil**, práctico e **incremental**. De esta manera, Grow se desarrolló mediante avances progresivos y revisiones constantes. El proyecto se trabajó mediante pequeñas etapas de desarrollo, en las cuales se implementaba una funcionalidad, se probaba su comportamiento y posteriormente se integraba al sistema. Este proceso permitió detectar errores, ajustar la estructura del código y mejorar la interfaz de usuario.

La metodología aplicada incluyó las siguientes prácticas:

- Desarrollo progresivo de funcionalidades.
- Construcción de prototipos funcionales.
- Revisión constante de errores.
- Uso de Trello para organizar tareas y avances.
- Uso de Git para registrar cambios en el proyecto.
- Uso de GitHub como repositorio remoto.
- Validación mediante pruebas unitarias y pruebas de widgets.
- Evaluación de rendimiento mediante Flutter DevTools.
- Análisis preventivo mediante árboles de fallos.
- Publicación de documentación mediante GitHub Pages.

Cabe destacar que, aunque Grow no implementa formalmente todos los elementos de Scrum, sí adopta una forma simplificada de trabajo ágil. Las tareas se organizaron en listas como pendiente, en progreso, en revisión, en pruebas y completado. Esto permitió controlar el avance del proyecto, identificar actividades pendientes y registrar el progreso de manera ordenada.

La metodología fue adecuada para el proyecto porque permitió construir una aplicación funcional en etapas, corregir problemas conforme aparecían y documentar el proceso de desarrollo de forma clara.

25 Entregables del proyecto

Los entregables del proyecto **Grow** corresponden a los productos generados durante el proceso de desarrollo, prueba, documentación y publicación. Los principales entregables del proyecto Grow son los siguientes:

- **Aplicación de escritorio Grow**

Aplicación funcional desarrollada con Flutter Desktop para Windows. Permite registrar, consultar, filtrar, analizar y exportar gastos personales.

- **Código fuente del proyecto**

Conjunto de archivos que conforman la aplicación, incluyendo pantallas, modelos, servicios, base de datos, pruebas y configuración del proyecto.

- **Base de datos local SQLite**

Estructura de almacenamiento local utilizada para guardar los gastos registrados por el usuario.

- **Pantallas principales del sistema**

Incluye la pantalla principal, pantalla de registro de gastos y pantalla de estadísticas.

- **Servicios de exportación**

Componentes encargados de generar reportes en formato PDF y Excel.

- **Pruebas unitarias**

Pruebas realizadas con Flutter Test para validar funciones lógicas del sistema, especialmente cálculos estadísticos.

- **Pruebas de widgets**

Pruebas realizadas para validar elementos principales de la interfaz, como la pantalla de registro de gastos y la pantalla principal.

- **Prueba de rendimiento**

Evaluación realizada con Flutter DevTools en modo profile para observar el comportamiento general de la aplicación durante su ejecución.

- **Árboles de fallos**

Análisis cualitativos realizados con ALD Free Web Fault Tree Analysis Tool para identificar posibles causas de eventos críticos, como errores en el registro de gastos, fallos en la generación de reportes PDF y cierre inesperado de la aplicación.

- **Repositorio en GitHub**

Repositorio remoto utilizado para almacenar, respaldar y controlar las versiones del proyecto.

- **Sitio web en GitHub Pages**

Página web pública utilizada para presentar la documentación básica del proyecto Grow.

- **Documento técnico**

Documento formal que describe la vista general del proyecto, funcionalidades, herramientas, arquitectura, pruebas, riesgos, evidencias y conclusiones.

- **Evidencias del proyecto**

Capturas de pantalla y registros que demuestran el funcionamiento de la aplicación, la ejecución de pruebas, el análisis de rendimiento, los árboles de fallos y la publicación del sitio web.

3 Organización del proyecto

La organización del proyecto **Grow** se estructura de acuerdo a las necesidades propias de una aplicación de escritorio desarrollada en un contexto académico. Debido a que se trata de un proyecto individual y, por ende, de alcance reducido, la totalidad de las actividades de análisis, diseño, programación, pruebas, documentación y publicación fueron realizadas por la desarrolladora principal del sistema. Aun cuando el proyecto no cuenta con una estructura empresarial formal, sí se identifican diferentes áreas de trabajo necesarias para su desarrollo. Estas áreas permiten organizar las responsabilidades del proyecto y facilitar el seguimiento de las actividades realizadas. La organización del proyecto se divide en las siguientes áreas principales:

- Planeación y análisis
- Diseño técnico
- Desarrollo de software
- Gestión de base de datos
- Pruebas y validación
- Documentación técnica
- Control de versiones
- Publicación del sitio web

3.1 Recursos humanos

Los recursos humanos del proyecto **Grow** están conformados principalmente por la desarrolladora del sistema quien asume la responsabilidad de construir, probar, documentar y presentar la aplicación. Además, intervienen actores académicos relacionados con la revisión y evaluación del proyecto, por lo cual, los recursos humanos pueden organizarse de la siguiente manera:

Área	Recurso humano	Función principal
Dirección académica	Docente o evaluador del proyecto	Revisar los avances, establecer criterios de entrega y evaluar el cumplimiento de los objetivos.
Desarrollo	Desarrolladora del proyecto	Analizar, diseñar, programar, probar y documentar la aplicación Grow.
Gestión del proyecto	Desarrolladora del proyecto	Organizar actividades, registrar avances y dar seguimiento mediante Trello y GitHub.
Diseño de interfaz	Desarrolladora del proyecto	Diseñar la apariencia visual de la aplicación, pantallas principales, navegación y logo.
Base de datos	Desarrolladora del proyecto	Implementar el almacenamiento local mediante SQLite.
Pruebas y calidad	Desarrolladora del proyecto	Ejecutar pruebas unitarias, pruebas de widgets, pruebas de rendimiento y análisis de fallos.
Usuario final	Persona que utiliza la aplicación	Registrar gastos, consultar información, filtrar datos y generar reportes.

3.2 Roles y responsabilidades

Para organizar el desarrollo de **Grow** se definieron distintos roles funcionales. Aunque una misma persona puede asumir más de un rol, esta división permite identificar con claridad las responsabilidades necesarias para completar el proyecto.

- **Responsable del proyecto**

Es la persona encargada de planear, coordinar y supervisar el avance general de Grow. Sus responsabilidades principales son:

- ✓ Definir el alcance del sistema.
- ✓ Organizar las actividades del proyecto.
- ✓ Dar seguimiento a los avances.
- ✓ Verificar el cumplimiento de los objetivos.
- ✓ Integrar las evidencias del desarrollo.
- ✓ Preparar la entrega final del proyecto.

- **Desarrolladora**

Responsable de construir la aplicación de escritorio. Sus actividades incluyen:

- ✓ Crear la estructura del proyecto Flutter.
- ✓ Programar las pantallas principales.
- ✓ Implementar el registro de gastos.
- ✓ Configurar la base de datos SQLite.
- ✓ Desarrollar filtros, estadísticas y reportes.
- ✓ Corregir errores detectados durante las pruebas.

- **Diseñadora de interfaz**

Encargada de la apariencia visual y experiencia básica de uso de la aplicación. Sus responsabilidades son:

- ✓ Diseñar la pantalla principal.
- ✓ Diseñar el formulario de registro de gastos.
- ✓ Diseñar la pantalla de estadísticas.
- ✓ Integrar el logo de Grow.
- ✓ Mejorar colores, botones, tarjetas y navegación.
- ✓ Procurar que la interfaz sea clara y sencilla para el usuario.

- **Responsable de base de datos**

Se enfoca en la estructura y funcionamiento del almacenamiento local. Sus responsabilidades son:

- ✓ Definir la estructura de la tabla de gastos.
- ✓ Implementar la conexión con SQLite.
- ✓ Guardar los gastos registrados.
- ✓ Consultar los gastos almacenados.
- ✓ Eliminar registros cuando sea necesario.
- ✓ Verificar que los datos se conserven correctamente.

- **Responsable de pruebas y calidad**

Tiene como finalidad comprobar que la aplicación funcione correctamente. Sus responsabilidades son:

- ✓ Diseñar y ejecutar pruebas unitarias.
- ✓ Realizar pruebas de widgets.
- ✓ Verificar la pantalla principal y el formulario de registro.
- ✓ Evaluar el rendimiento con Flutter DevTools.
- ✓ Identificar posibles fallos críticos mediante árboles de fallos.
- ✓ Documentar los resultados obtenidos.

- **Responsable de documentación**

Se encarga de registrar el proceso de desarrollo y preparar los documentos necesarios para la entrega del proyecto. Sus responsabilidades son:

- ✓ Elaborar el documento técnico.
- ✓ Redactar el manual básico de uso.
- ✓ Registrar herramientas utilizadas.
- ✓ Incluir evidencias de pruebas y validación.
- ✓ Documentar el control de versiones.
- ✓ Actualizar el sitio web publicado en GitHub Pages.

- **Usuario final**

Es la persona que utilizará Grow para administrar sus gastos personales. Sus acciones principales dentro del sistema son:

- ✓ Registrar gastos personales.
- ✓ Consultar la lista de gastos.
- ✓ Filtrar gastos por categoría.
- ✓ Revisar estadísticas.
- ✓ Exportar reportes en PDF o Excel.

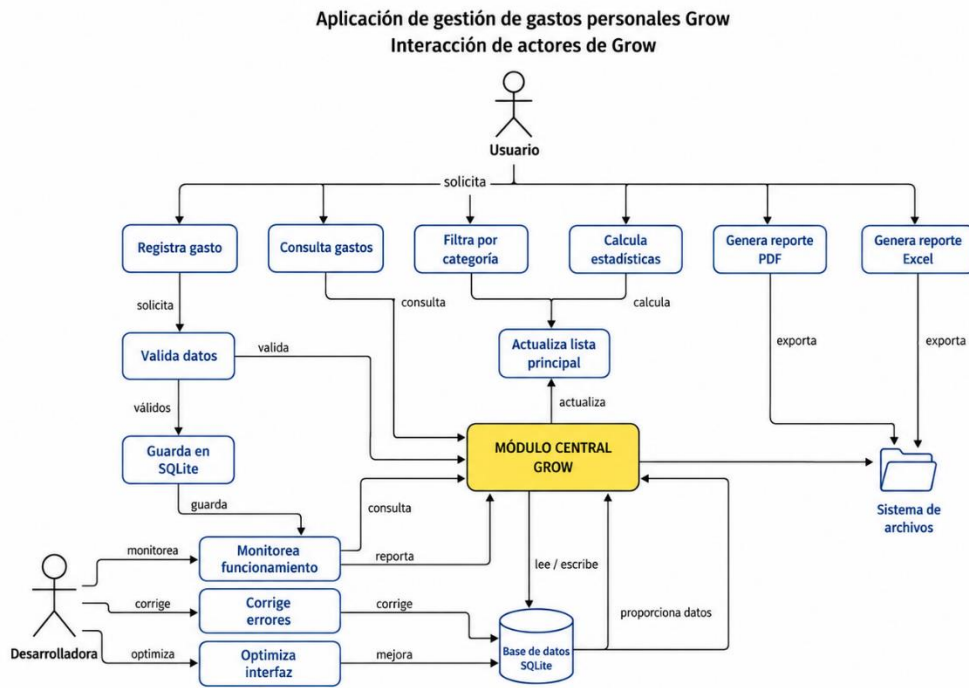


Ilustración. Interacción entre actores y módulos de Grow

Ilustración 1. Interacciones entre actores en Grow

4 Gestión de procesos en el proyecto

La gestión de procesos del proyecto **Grow** comprende la planeación, organización, seguimiento, control y configuración de las actividades necesarias para desarrollar la aplicación de escritorio de gestión de gastos personales. Esta gestión permite ordenar las tareas del proyecto, establecer fechas de entrega, controlar avances, registrar cambios y verificar que los entregables cumplan con los objetivos establecidos. El proyecto fue organizado en fases progresivas, desde la revisión de requerimientos y definición del alcance hasta la entrega final del sistema.

La gestión de procesos se apoyó en herramientas como **Trello**, para organizar actividades; **Git**, para registrar cambios locales; **GitHub**, para respaldar el proyecto; y **GitHub Pages**, para publicar la documentación web. Además, se utilizaron herramientas de validación como **Flutter Test**, **Flutter DevTools** y **ALD Free Web Fault Tree Analysis Tool**.

4.1 Plan de actividades

El plan de actividades de **Grow** se organizó por semanas, considerando las principales fases del desarrollo: planeación, diseño técnico, preparación del entorno, diseño de interfaz, desarrollo funcional, pruebas, validación, documentación y entrega final. La tabla de actividades del proyecto establece una secuencia de trabajo que inicia con la revisión del documento SRS, la definición del alcance y la organización de actividades durante la primera semana. Posteriormente, incluye actividades de diseño de arquitectura, configuración del entorno, diseño de interfaz, diagramas UML, desarrollo de módulos, pruebas, publicación del sitio web y cierre del proyecto.

La siguiente tabla muestra las actividades realizadas en un período cuya duración fue de 10 semanas desde el inicio hasta la conclusión del proyecto hasta el momento:

Semana	Actividad	Clasificación	Fecha de entrega
1	Revisión del documento SRS y definición del alcance final del proyecto	Planeación y análisis	21/may/2026
1	Organización de las actividades	Gestión del proyecto	25/may/2026
2	Diseño de arquitectura del sistema y estructura modular	Diseño técnico	31/may/2026
3	Configuración del entorno de desarrollo en Flutter, Firebase y SQLite	Preparación técnica	07/jun/2026

Semana	Actividad	Clasificación	Fecha de entrega
4	Diseño de interfaz gráfica (pantallas principales y navegación)	Diseño UI/UX	14/jun/2026
4	Elaboración y validación de diagramas UML finales	Modelado del sistema	14/jun/2026
5	Desarrollo del módulo de registro de gastos	Desarrollo frontend/backend	21/jun/2026
5	Implementación de validaciones y almacenamiento local en SQLite	Desarrollo backend	21/jun/2026
6	Desarrollo del módulo de estadísticas y gráficos interactivos	Desarrollo funcional	28/jun/2026
6	Implementación de filtros de búsqueda y consultas	Desarrollo funcional	28/jun/2026
7	Desarrollo del módulo de exportación de reportes PDF y Excel	Desarrollo funcional	05/jul/2026
8	Pruebas funcionales, pruebas de interfaz y corrección de errores	Testing y QA	19/jul/2026
8	Validación de requisitos no funcionales	Testing y QA	19/jul/2026
9	Elaboración de sitio web en GitHub Pages	Cierre del proyecto	26/jul/2026
10	Ajustes finales, documentación técnica y manual de usuario	Cierre del proyecto	05/ago/2026
10	Entrega final y presentación del sistema Grow	Implementación y cierre	05/ago/2026

42 Seguimiento y control

• Seguimiento de actividades

Se realizó mediante la organización del trabajo en tareas específicas. Para ello se utilizó **Trello**, donde las actividades se clasificaron en listas como pendiente, en progreso, en revisión, en pruebas y completado. Este mecanismo permitió visualizar el estado de cada tarea, identificar actividades terminadas y reconocer aquellas que aún requerían atención. Entre las tareas controladas se incluyeron el desarrollo de la aplicación, la implementación de filtros, estadísticas, exportación PDF y Excel, pruebas, análisis de rendimiento, árboles de fallos y documentación.

• Control de plazos

El control de plazos se basó en las fechas de entrega definidas en la tabla de actividades. Estas fechas permitieron establecer metas semanales y organizar el avance progresivo del proyecto. En el siguiente diagrama se despliegan dichas actividades en un diagrama de Gantt:

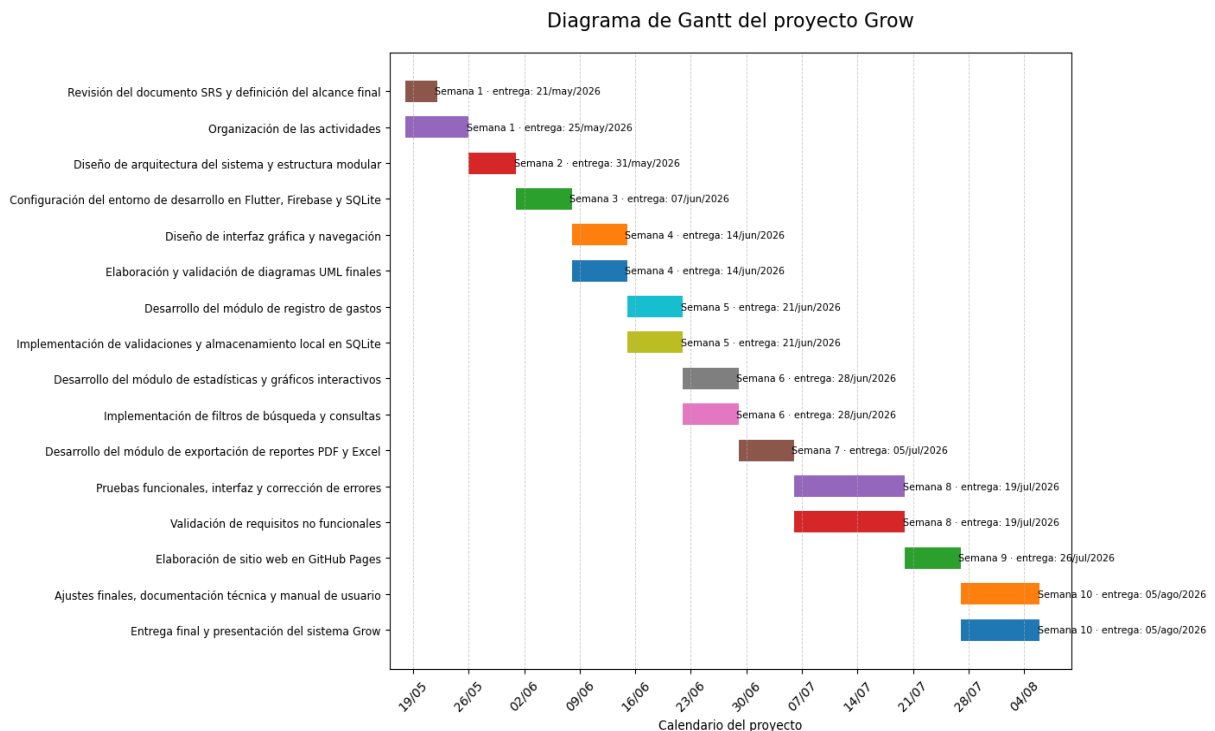


Ilustración 2. Diagrama de Gantt de Grow

- **Control de calidad**

El control de calidad se llevó a cabo mediante la ejecución de pruebas y validaciones sobre la aplicación. En Grow se aplicaron pruebas unitarias, pruebas de widgets, pruebas de rendimiento y análisis de fallos. Las pruebas unitarias permitieron verificar funciones lógicas, como el cálculo de totales, promedios y gastos por categoría y se ejecutaron con **Flutter Test** al igual que las pruebas de widgets permitieron comprobar elementos visuales y funcionales de la interfaz. Las pruebas de rendimiento con **Flutter DevTools** permitieron observar el comportamiento general de la aplicación durante su ejecución. Finalmente, los árboles de fallos permitieron analizar posibles eventos críticos de forma preventiva.

- **Gestión de riesgos**

La gestión de riesgos consistió en identificar posibles fallos que pudieran afectar el funcionamiento de Grow. Para ello se elaboraron árboles de fallos con la herramienta **ALD Free Web Fault Tree Analysis Tool**. Los eventos críticos analizados fueron:

- El gasto válido no se registra o no aparece en la lista.
- El reporte PDF no se genera correctamente.
- La aplicación se cierra inesperadamente.

- **Control de cambios**

El control de cambios se realizó mediante **Git** y **GitHub**. Cada avance importante del proyecto fue registrado mediante commits, lo cual permitió conservar un historial de modificaciones y respaldar el código fuente en un repositorio remoto. Entre los cambios registrados se incluyen la configuración inicial del proyecto, implementación de SQLite, pantalla de estadísticas, filtros, exportación PDF y Excel, mejoras visuales, integración del logo, pruebas y publicación del sitio web.

4.3 Gestión de configuración

La gestión de configuración de **Grow** tuvo como finalidad controlar los elementos técnicos del proyecto, registrar versiones, administrar cambios y mantener organizada la estructura del sistema mediante el uso de Git, GitHub y una estructura organizada de carpetas y archivos.

- **Elementos bajo configuración**

Los principales elementos bajo gestión de configuración fueron:

- Código fuente de la aplicación.
- Archivos de configuración del proyecto Flutter.
- Base de datos local SQLite.
- Servicios de exportación PDF y Excel.
- Pantallas de la aplicación.
- Pruebas unitarias y pruebas de widgets.
- Archivos del sitio web en GitHub Pages.
- Logo e imágenes del proyecto.
- Documento técnico y evidencias.

- **Estructura del proyecto**

La estructura principal del proyecto Grow se organizó de la siguiente manera:

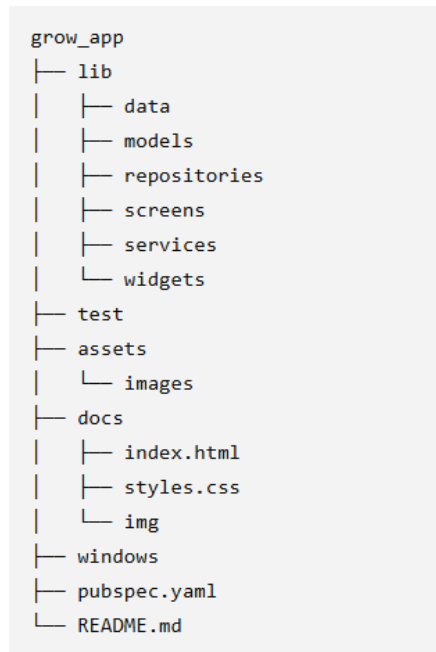


Ilustración 3. Estructura de Grow

- **Control de versiones**

El control de versiones se realizó con **Git**. Cada cambio importante fue agregado al control de versiones y guardado mediante commits descriptivos. Posteriormente, los cambios fueron enviados a GitHub mediante *git push*. Este proceso permitió:

- ✓ Registrar el historial de cambios.
- ✓ Respalidar el proyecto en la nube.
- ✓ Recuperar versiones anteriores si fuera necesario.
- ✓ Documentar avances importantes.
- ✓ Publicar el sitio web del proyecto mediante GitHub Pages.

- **Repositorio remoto**

El repositorio remoto del proyecto se alojó en **GitHub**. Este repositorio contiene el código fuente de la aplicación, archivos de configuración, pruebas, documentación y sitio web. El uso de GitHub permitió mantener una copia remota del proyecto y facilitó la publicación del sitio de documentación mediante GitHub Pages.

El enlace del repositorio remoto es: https://github.com/a374279-debug/grow_app

- **Publicación de documentación**

La documentación web del proyecto se publicó mediante **GitHub Pages**. Para ello se utilizó la carpeta docs, donde se almacenaron los archivos *index.html*, *styles.css* y las imágenes utilizadas en el sitio. El sitio web permite presentar de forma pública información general sobre **Grow**, sus funcionalidades principales, herramientas utilizadas, manual básico de uso, estructura del proyecto, estado actual y conclusión.

El enlace del sitio web es: https://a374279-debug.github.io/grow_app/

- **Línea base del proyecto**

La línea base del proyecto se conformó a partir de las versiones funcionales que fueron guardadas en GitHub. Cada avance estable representó una versión de referencia del sistema. Entre las líneas base más importantes del proyecto se encuentran:

- Configuración inicial del proyecto Flutter.
- Implementación del registro de gastos.
- Integración de SQLite.
- Incorporación de filtros por categoría.
- Desarrollo de estadísticas.
- Exportación de reportes PDF y Excel.
- Creación de pruebas unitarias y de widgets.
- Publicación del sitio web en GitHub Pages.
- Actualización del logo y documentación del proyecto.

5 Infraestructura

La infraestructura del proyecto **Grow** está conformada por los recursos de hardware, software y herramientas que fueron usadas para desarrollar, ejecutar, probar, documentar y publicar la aplicación en cuestión. A diferencia de un sistema web cliente/servidor, Grow funciona como una aplicación de escritorio local para Windows por lo que la mayor parte de su infraestructura se concentra en el equipo donde se desarrolla y ejecuta el sistema. El proyecto utiliza **Flutter Desktop** como tecnología principal para la construcción de la aplicación, **Dart** como lenguaje de programación y **SQLite** como sistema de almacenamiento local. Asimismo, se emplean herramientas complementarias para el control de versiones, pruebas, validación, rendimiento, documentación y publicación web.

La infraestructura del proyecto puede dividirse en tres apartados principales:

- Hardware
- Software
- Herramientas utilizadas

5.1 Hardware

Para el desarrollo y ejecución de Grow se requiere un equipo de cómputo capaz de ejecutar aplicaciones de escritorio desarrolladas con Flutter. Debido a que Grow es una aplicación local, no requiere servidor externo, infraestructura en la nube ni conexión permanente a Internet para registrar, consultar o analizar gastos.

Equipo de desarrollo

El equipo de desarrollo debe contar con las características suficientes para ejecutar Visual Studio Code, Flutter SDK, Dart, emuladores o herramientas de depuración, y la aplicación de escritorio en Windows.

Los requerimientos recomendados son los siguientes:

Recurso	Característica recomendada
Equipo	Computadora de escritorio o laptop
Sistema operativo	Windows 10 o superior
Procesador	Intel Core i3, Ryzen 3 o superior
Memoria RAM	8 GB recomendados
Almacenamiento	5 GB libres como mínimo para proyecto, SDK, dependencias y reportes
Pantalla	Resolución mínima recomendada de 1366 x 768
Conectividad	Internet para instalar dependencias, usar GitHub y publicar documentación

Equipo de usuario final

El usuario final necesita un equipo Windows donde pueda ejecutarse la aplicación Grow. Ya que la información se guarda localmente, el usuario no necesita conexión a Internet para usar las funciones principales de registro, consulta, filtrado y generación de estadísticas.

Los requerimientos recomendados para el usuario final son:

Recurso	Característica recomendada
Equipo	Computadora de escritorio o laptop
Sistema operativo	Windows 10 o superior
Memoria RAM	4 GB como mínimo
Almacenamiento	Espacio disponible para la base de datos local y reportes generados
Conectividad	No requerida para uso local; necesaria solo para descargar actualizaciones o consultar el sitio web

Almacenamiento local

Grow hace uso de almacenamiento local para guardar la base de datos SQLite y los reportes exportados. Los archivos generados por la aplicación, como reportes PDF y Excel, se almacenan en el equipo del usuario por lo cual es necesario contar con los permisos adecuados de escritura en la carpeta correspondiente.

5.2 Software

El software requerido para el desarrollo y funcionamiento de Grow se divide en software de desarrollo, software de ejecución y software de documentación.

Software de desarrollo

Software	Uso en el proyecto
Windows	Sistema operativo utilizado para desarrollar y ejecutar la aplicación de escritorio
Flutter SDK	Framework utilizado para construir la aplicación de escritorio
Dart SDK	Lenguaje y entorno de ejecución utilizado por Flutter
Visual Studio Code	Editor de código utilizado para programar, organizar archivos y ejecutar comandos
Git	Sistema de control de versiones utilizado para registrar cambios del proyecto
GitHub	Plataforma remota utilizada para almacenar el repositorio del proyecto

Software de ejecución

Software	Uso en el proyecto
Flutter Desktop para Windows	Permite generar y ejecutar la aplicación como programa de escritorio
SQLite	Base de datos local utilizada para almacenar los gastos registrados
sqlite_common_ffi	Dependencia utilizada para trabajar con SQLite en entorno de escritorio
Librerías de exportación PDF	Permiten generar reportes de gastos en formato PDF
Librerías de exportación Excel	Permiten generar reportes de gastos en formato Excel

Software de documentación y publicación

Software o servicio	Uso en el proyecto
Microsoft Word	Elaboración del documento técnico del proyecto
GitHub Pages	Publicación del sitio web de documentación
Navegador web	Visualización del sitio publicado y revisión de la documentación web
ALD Free Web Fault Tree Analysis Tool	Elaboración de árboles de fallos
Flutter DevTools	Evaluación del rendimiento de la aplicación
Flutter Test	Ejecución de pruebas unitarias y pruebas de widgets

5.3 Herramientas utilizadas

Las herramientas utilizadas en **Grow** fueron seleccionadas de acuerdo con las necesidades técnicas del proyecto y su alcance académico; cada una de las herramientas cumplió una función específica dentro del desarrollo, validación o documentación del sistema.

- **Flutter Desktop**

Fue la tecnología principal usada en el desarrollo de esta aplicación de escritorio. Permitted construir la interfaz gráfica, manejar la navegación entre pantallas y organizar los componentes visuales mediante widgets. Se utilizó para desarrollar:

- ✓ Pantalla principal
- ✓ Pantalla de registro de gastos
- ✓ Pantalla de estadísticas
- ✓ Botones de exportación
- ✓ Filtros por categoría
- ✓ Diseño visual de tarjetas, encabezado y logo

- **Dart**

Fue el lenguaje de programación utilizado para implementar la lógica de la aplicación. Con Dart se desarrollaron las clases, servicios, validaciones, cálculos y funciones necesarias para que Grow pudiera registrar, consultar, filtrar y exportar gastos.

- **SQLite**

Fue utilizado como sistema de base de datos local. Su función principal fue almacenar los gastos registrados por el usuario sin necesidad de conexión a Internet. En este proyecto, SQLite permite:

- ✓ Guardar gastos
- ✓ Consultar gastos registrados
- ✓ Eliminar registros
- ✓ Ordenar información por fecha
- ✓ Conservar los datos localmente

- **sqflite_common_ffi**

La dependencia en cuestión permitió usar SQLite en un entorno de escritorio; esta herramienta fue necesaria porque Grow se ejecuta como aplicación de escritorio en Windows.

- **Visual Studio Code**

Fue el editor principal del proyecto usado para escribir código, crear archivos, organizar carpetas, ejecutar comandos de Flutter, revisar errores y administrar el proyecto desde la terminal integrada.

- **Git**

Usado para controlar las versiones del proyecto en el que cada avance importante fue registrado mediante commits lo cual permitió mantener un historial de cambios y conservar versiones estables de la aplicación.

- **GitHub**

Se usó como repositorio remoto para almacenar el proyecto Grow; permitió respaldar el código fuente, consultar el historial de cambios y mantener disponible el proyecto en línea.

- **GitHub Pages**

Usado para publicar el sitio web de documentación del proyecto. El sitio incluye información general sobre Grow, sus funcionalidades, herramientas utilizadas, estructura del proyecto, estado actual y conclusión.

- **Trello**

Se usó como herramienta de organización del proyecto; permitió clasificar actividades en listas como pendiente, en progreso, en revisión, en pruebas y completado.

- **Flutter Test**

Se utilizó para realizar pruebas unitarias y pruebas de widgets; con esta herramienta se validaron funciones lógicas del sistema y elementos principales de la interfaz.

- **Flutter DevTools**

Usado para evaluar el rendimiento de la aplicación en modo profile. Permitted observar el comportamiento de la interfaz, los frames y la estabilidad general del sistema durante la ejecución.

- **ALD Free Web Fault Tree Analysis Tool**

Usada para elaborar árboles de fallos; esta herramienta permitió analizar eventos críticos del sistema, como fallos en el registro de gastos, generación de reportes PDF y cierre inesperado de la aplicación.

- **Microsoft Word**

Se utilizó para elaborar el documento técnico del proyecto, integrar la información del desarrollo, organizar secciones, insertar tablas, evidencias, diagramas y redactar la documentación final.

6 Diseño de la arquitectura del sistema

El diseño de la arquitectura del sistema **Grow** describe la organización técnica de la aplicación, sus componentes principales, la forma en que se estructuran los archivos del proyecto y la manera en que la aplicación gestiona la información de los gastos personales.

A diferencia de sistemas cliente-servidor o aplicaciones web, Grow fue diseñado como una **aplicación de escritorio local**. Esto significa que la mayor parte de sus operaciones se ejecutan directamente en el equipo del usuario, sin depender de un servidor externo para registrar, consultar o analizar los datos. La información se almacena localmente mediante **SQLite**, lo cual permite conservar los gastos registrados aun cuando no exista conexión a Internet. La arquitectura de Grow se organiza de manera **modular**, esto es, cada parte del sistema cumple una función específica: la interfaz permite la interacción con el usuario, la capa de datos administra la conexión con SQLite, los modelos representan la información, los servicios generan cálculos o reportes, y las pantallas muestran las funcionalidades principales.

6.1 Descripción general

Grow fue desarrollado con **Flutter Desktop** y **Dart**, siguiendo una estructura modular que facilita la organización, mantenimiento y ampliación del sistema y permite construir una interfaz gráfica de escritorio, manejar componentes visuales mediante widgets y organizar la lógica del sistema en archivos y carpetas especializadas. La interacción principal inicia cuando el usuario registra un gasto mediante la interfaz. El sistema valida los datos ingresados, crea un objeto de tipo gasto y lo almacena en SQLite. Posteriormente, la pantalla principal consulta la base de datos y muestra los registros guardados. Además, el usuario puede filtrar los gastos por categoría, consultar estadísticas y exportar reportes.

La arquitectura general de Grow puede entenderse mediante las siguientes capas:

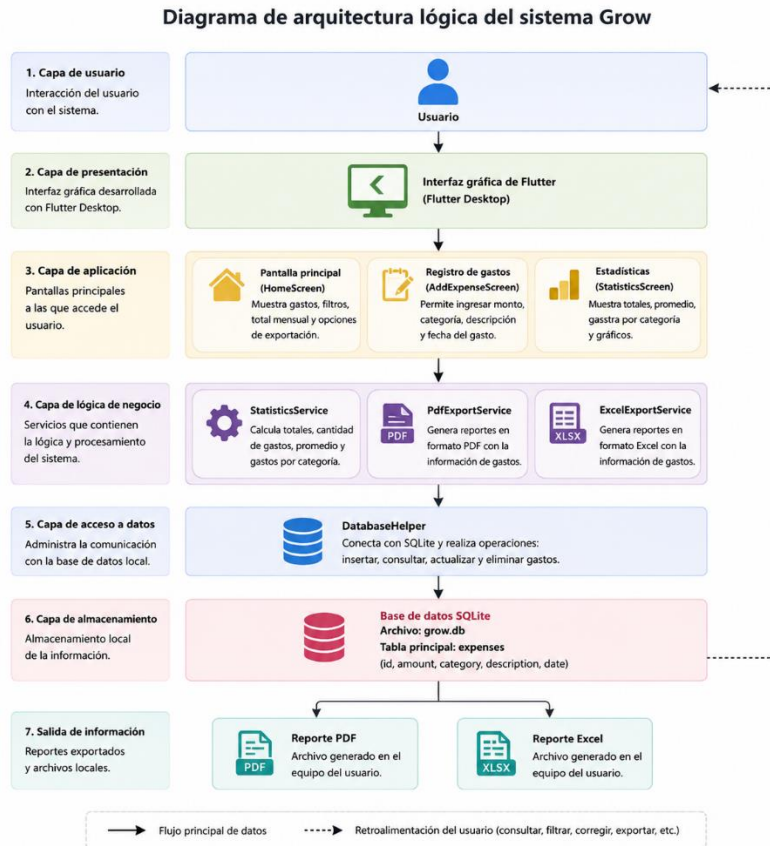


Ilustración 4. Diagrama de arquitectura lógica del sistema Grow

La arquitectura se diseñó para que el sistema fuera:

- **Local:** funciona en el equipo del usuario.
- **Modular:** separa pantallas, datos, modelos y servicios.
- **Mantenible:** permite modificar partes específicas sin afectar todo el sistema.
- **Extensible:** puede incorporar nuevas funciones en el futuro.
- **Funcional:** cubre las operaciones principales de gestión de gastos personales.

6.2 Estructura del proyecto

La estructura del proyecto Grow se organizó en carpetas que separan los elementos principales de la aplicación. Esta organización facilita la localización de archivos, el mantenimiento del código y la incorporación de nuevas funcionalidades.

La estructura general del proyecto es la siguiente:

```
grow_app
├── lib
│   ├── data
│   ├── models
│   ├── repositories
│   ├── screens
│   ├── services
│   └── widgets
├── test
├── assets
│   └── images
├── docs
│   ├── index.html
│   ├── styles.css
│   └── img
├── windows
├── pubspec.yaml
└── README.md
```

Ilustración 3. Estructura de Grow

La carpeta **lib** contiene el código principal de la aplicación; dentro de esta se organizan los archivos relacionados con la base de datos, los modelos, las pantallas, los servicios y los widgets.

La carpeta **data** contiene los archivos encargados de la conexión y administración de la base de datos SQLite. En esta carpeta se encuentra el archivo que permite insertar, consultar, actualizar o eliminar gastos.

La carpeta **models** contiene las clases que representan los datos del sistema; en Grow, el modelo principal es **Expense** el cual representa cada gasto registrado por el usuario.

La carpeta **screens** contiene las pantallas principales de la aplicación, como la pantalla de inicio, la pantalla para registrar gastos y la pantalla de estadísticas.

La carpeta **services** contiene funciones auxiliares y servicios especializados, como el cálculo de estadísticas y la generación de reportes PDF y Excel.

La carpeta **widgets** puede usarse para almacenar componentes visuales reutilizables dentro de la interfaz.

La carpeta **test** contiene las pruebas unitarias y pruebas de widgets desarrolladas con Flutter Test.

La carpeta **assets/images** contiene recursos visuales utilizados por la aplicación como el logo de **Grow**.

La carpeta **docs** contiene los archivos del sitio web publicado mediante GitHub Pages; dicha carpeta incluye el archivo index.html, el archivo styles.css y las imágenes utilizadas en la página web.

Finalmente, el archivo **pubspec.yaml** contiene la configuración general del proyecto Flutter, incluyendo dependencias, recursos e información básica de la aplicación.

6.3 Base de datos

Grow hace uso de **SQLite** como sistema de almacenamiento local; dicha base de datos permite guardar los gastos registrados por el usuario directamente en el equipo donde se ejecuta la aplicación. La base de datos principal del sistema se denomina **grow.db** y dentro de esta se crea una tabla llamada **expenses**.

La tabla **expenses** almacena la información básica de cada gasto. Su estructura general es la siguiente:

Campo	Tipo de dato	Descripción
id	INTEGER	Identificador único del gasto. Se genera automáticamente
amount	REAL	Monto del gasto registrado
category	TEXT	Categoría del gasto
description	TEXT	Descripción breve del gasto
date	TEXT	Fecha del gasto, almacenada en formato de texto

El uso de SQLite permite que la aplicación funcione sin conexión a Internet. Además, facilita operaciones básicas de almacenamiento, consulta y eliminación de información. Las operaciones principales que realiza Grow sobre la base de datos son:

- ✓ Insertar un nuevo gasto
- ✓ Consultar todos los gastos registrados
- ✓ Ordenar los gastos por fecha
- ✓ Eliminar un gasto
- ✓ Actualizar un gasto (cuando es necesario)

El componente encargado de administrar estas operaciones es **DatabaseHelper**, el cual establece la conexión con SQLite y contiene los métodos necesarios para trabajar con la tabla expenses. La información almacenada en SQLite se utiliza posteriormente para mostrar la lista de gastos, calcular totales, generar estadísticas, aplicar filtros y exportar reportes.

6.4 Componentes principales

La arquitectura de **Grow** se compone de diversos elementos que trabajan en conjunto para permitir el funcionamiento completo de la aplicación. Sus se organizan de forma modular dentro del proyecto Flutter.

Modelo Expense

El modelo Expense representa un gasto dentro del sistema; también permite convertir la información del gasto en un mapa de datos para guardarlo en SQLite y reconstruir el objeto cuando se consulta desde la base de datos. Contiene los datos principales que el usuario registra:

- Identificador
- Monto
- Categoría
- Descripción
- Fecha

DatabaseHelper

Es el componente encargado de administrar la base de datos local; actúa como intermediario entre la aplicación y la base de datos. Sus responsabilidades principales son:

- Inicializar SQLite
- Crear la base de datos grow.db
- Crear la tabla expenses
- Insertar gastos
- Consultar gastos
- Eliminar gastos
- Actualizar gastos

HomeScreen

Corresponde a la pantalla principal de Grow; desde esta pantalla el usuario puede visualizar los gastos registrados, consultar el total mensual, aplicar filtros por categoría, acceder a estadísticas y exportar reportes. Sus funciones principales son:

- Mostrar el encabezado de Grow
- Mostrar el total mensual
- Mostrar la lista de gastos
- Aplicar filtro por categoría
- Permitir eliminar gastos
- Acceder al registro de gastos
- Acceder a estadísticas
- Exportar reportes PDF y Excel

AddExpenseScreen

Es la pantalla encargada del registro de nuevos gastos y donde el usuario ingresa el monto, categoría, descripción y fecha del gasto. Sus responsabilidades principales son:

- Mostrar el formulario de registro
- Validar que el monto sea correcto
- Validar que la descripción no esté vacía
- Permitir seleccionar categoría
- Permitir seleccionar fecha
- Guardar el gasto en SQLite

StatisticsScreen

Es la pantalla de estadísticas y cuya función es presentar información general sobre los gastos registrados. Esta pantalla permite visualizar:

- Total de gastos
- Cantidad de gastos registrados
- Promedio de gastos
- Gasto por categoría
- Gráficos estadísticos

StatisticsService

Contiene la lógica que se necesita para calcular información estadística; dicho componente permite separar los cálculos de la interfaz lo cual facilita realizar pruebas unitarias. Sus funciones principales son:

- Calcular el total de gastos
- Contar la cantidad de gastos
- Calcular el promedio
- Agrupar gastos por categoría

PdfExportService

Es el servicio encargado de generar reportes en formato PDF; toma la información de los gastos y la organiza en un documento exportable. Sus responsabilidades son:

- Recibir la lista de gastos
- Crear un documento PDF
- Organizar la información en formato de reporte
- Calcular totales
- Guardar el archivo en el sistema

ExcelExportService

Es el servicio encargado de generar reportes en formato Excel y cuya función es convertir la información de los gastos en una hoja de cálculo que pueda consultarse posteriormente. Sus responsabilidades son:

- Crear un archivo Excel
- Insertar encabezados
- Registrar gastos en filas
- Agregar información de totales

- Guardar el archivo generado

Sitio web de documentación

Esta forma parte de los componentes complementarios del proyecto; se desarrolló con HTML y CSS dentro de la carpeta docs, y publicado mediante GitHub Pages.

Permite presentar:

- Descripción del proyecto
- Funcionalidades principales
- Herramientas utilizadas
- Manual básico de uso
- Estado actual del proyecto
- Conclusión

Pruebas del sistema

Forman parte de la arquitectura de calidad del proyecto y se encuentran dentro de la carpeta **test**; permiten comprobar que algunos componentes funcionen correctamente. Las pruebas realizadas incluyen:

- Pruebas unitarias para StatisticsService
- Pruebas de widgets para AddExpenseScreen
- Prueba general de interfaz para la pantalla principal

7 Ingeniería de requerimientos

La ingeniería de requerimientos del proyecto **Grow** comprende la identificación, clasificación y descripción de las necesidades que debe satisfacer la aplicación de escritorio para la gestión de gastos personales. Esta sección permite establecer qué funciones debe realizar el sistema, bajo cuáles condiciones debe operar y cuáles son sus principales limitaciones. Los requisitos (o requerimientos) de Grow fueron definidos a partir del objetivo general del proyecto: desarrollar una aplicación local que permita registrar, consultar, filtrar, analizar y exportar información relacionada con los gastos personales del usuario.

Para organizar los requisitos, se clasifican en tres grupos principales:

- Requerimientos funcionales
- Requerimientos no funcionales
- Restricciones

7.1 Requerimientos funcionales

Los requerimientos funcionales describen las funciones principales que la aplicación Grow debe ofrecer al usuario; dichos requisitos están relacionados directamente con las operaciones que permiten registrar, administrar, consultar y exportar gastos personales. Estos requisitos permiten cubrir el flujo principal de uso de Grow. El usuario registra un gasto, el sistema valida la información, la almacena de manera local y posteriormente permite consultarla, filtrarla, analizarla o exportarla. Los requisitos más críticos son aquellos relacionados con el registro, almacenamiento, consulta y exportación de gastos.

La tabla siguiente los despliega para su fácil visualización:

Código	Requisito funcional	Descripción	Prioridad
RF-01	Registrar gastos personales	El sistema debe permitir registrar un gasto con monto, categoría, descripción y fecha.	Alta
RF-02	Validar datos del formulario	El sistema debe verificar que el monto sea válido y que los campos requeridos no estén vacíos.	Alta
RF-03	Almacenar gastos en SQLite	El sistema debe guardar los gastos registrados en una base de datos local SQLite.	Alta
RF-04	Consultar gastos registrados	El sistema debe mostrar en la pantalla principal la lista de gastos almacenados.	Alta
RF-05	Eliminar gastos	El sistema debe permitir eliminar un gasto registrado cuando el usuario lo requiera.	Media
RF-06	Filtrar gastos por categoría	El sistema debe permitir visualizar gastos según la categoría seleccionada.	Alta
RF-07	Calcular total mensual	El sistema debe calcular y mostrar el total de gastos correspondientes al mes actual.	Alta
RF-08	Generar estadísticas	El sistema debe calcular información como total, cantidad de gastos, promedio y gastos por categoría.	Alta
RF-09	Mostrar gráficos estadísticos	El sistema debe representar visualmente los gastos por categoría mediante gráficos.	Media
RF-10	Exportar reporte PDF	El sistema debe permitir generar un reporte de gastos en formato PDF.	Alta
RF-11	Exportar reporte Excel	El sistema debe permitir generar un reporte de gastos en formato Excel.	Alta
RF-12	Publicar documentación web	El proyecto debe contar con un sitio web de documentación publicado mediante GitHub Pages.	Media

7.2 Requerimientos no funcionales

Describen las condiciones de calidad que debe cumplir Grow para funcionar de manera adecuada. Estos requisitos no representan acciones directas del usuario, sino características esperadas del sistema. Los requisitos no funcionales más importantes para Grow son la usabilidad, la confiabilidad del almacenamiento local y el rendimiento general de la aplicación pues estos elementos determinan que el usuario pueda registrar y consultar sus gastos sin errores visibles o pérdida de información.

Se muestran en la siguiente tabla:

Código	Requisito no funcional	Descripción	Prioridad
RNF-01	Usabilidad	La interfaz debe ser clara, sencilla y fácil de utilizar para registrar y consultar gastos.	Alta
RNF-02	Rendimiento	La aplicación debe responder de forma estable durante el registro, consulta, filtrado y exportación de información.	Alta
RNF-03	Confiabilidad	Los gastos registrados deben conservarse correctamente en la base de datos local.	Alta
RNF-04	Disponibilidad local	La aplicación debe poder utilizarse sin conexión a Internet para sus funciones principales.	Alta
RNF-05	Mantenibilidad	El código debe estar organizado en carpetas y componentes que faciliten futuras modificaciones.	Media
RNF-06	Compatibilidad	El sistema debe ejecutarse como aplicación de escritorio en Windows.	Alta
RNF-07	Seguridad básica de datos	La información debe almacenarse localmente en el equipo del usuario.	Media
RNF-08	Portabilidad del proyecto	El código fuente debe poder respaldarse y consultarse desde GitHub.	Media
RNF-09	Calidad software del	El sistema debe ser validado mediante pruebas unitarias, pruebas de widgets y pruebas de rendimiento.	Alta
RNF-10	Documentación	El proyecto debe contar con documentación técnica y sitio web informativo.	Alta

7.3 Restricciones

Las restricciones definen los límites técnicos, funcionales y académicos bajo los cuales se desarrolló Grow. Estas condiciones permiten delimitar el alcance real del sistema y establecer qué elementos no forman parte de la versión actual. Grow cumple con las funciones principales de una aplicación local para gestión de gastos personales, pero no incluye características avanzadas como cuentas de usuario, sincronización remota, respaldo en la nube, integración bancaria o funcionamiento multiplataforma completo.

En futuras versiones, algunas de estas restricciones podrían convertirse en oportunidades de mejora, por ejemplo, agregar respaldo en la nube, autenticación, versión móvil o sincronización entre dispositivos.

La siguiente tabla muestra las restricciones del sistema:

Código	Restricción	Descripción
RES-01	Aplicación de escritorio	Grow está diseñado como aplicación de escritorio para Windows.
RES-02	Almacenamiento local	La información se guarda localmente mediante SQLite.
RES-03	Sin sincronización en la nube	El sistema no incluye respaldo automático ni sincronización con servidores externos.
RES-04	Sin autenticación de usuarios	La versión actual no incluye inicio de sesión ni manejo de múltiples cuentas.
RES-05	Sin conexión bancaria	La aplicación no se conecta con bancos, tarjetas o servicios financieros externos.
RES-06	Dependencia de Flutter Desktop	El desarrollo y ejecución dependen de la correcta configuración de Flutter para escritorio.
RES-07	Dependencia de paquetes externos	La exportación PDF y Excel depende del funcionamiento de las librerías utilizadas.
RES-08	Alcance académico	El sistema fue desarrollado como proyecto académico, por lo que no sustituye una herramienta financiera profesional.
RES-09	Análisis de fallos cualitativo	Los árboles de fallos no incluyen cálculos probabilísticos debido a la ausencia de datos históricos reales.
RES-10	Plataforma objetivo limitada	La versión actual se enfoca en Windows y no contempla versión móvil o web funcional.

8 Procesos del sistema

Estos describen las interacciones principales entre los actores y la aplicación. Permiten comprender cómo el usuario hace uso del sistema, qué acciones puede realizar y cómo la aplicación responde ante cada solicitud; se organizan a partir de las funciones principales de la aplicación: registrar gastos, consultar información, filtrar registros, generar estadísticas, exportar reportes y administrar el funcionamiento general del sistema. De modo que Grow es una aplicación de escritorio local, sus procesos son más sencillos; la mayoría de las acciones son iniciadas por el usuario desde la interfaz gráfica y ejecutadas por la aplicación mediante sus componentes internos, como las pantallas, servicios, base de datos SQLite y módulos de exportación.

8.1 Actores

Son los usuarios o elementos que interactúan de forma directa o indirecta con el sistema. En el caso de Grow, se identifican actores humanos y componentes internos que participan en el funcionamiento de la aplicación. En términos generales, puede decirse que el actor principal es el **usuario final** pues es quien inicia la mayoría de los procesos del sistema; el usuario interactúa con Grow mediante la interfaz gráfica, selecciona opciones, registra información y solicita consultas o reportes.

Por otra parte, la **desarrolladora** participa como actor técnico de modo que se encarga de la implementación, pruebas, correcciones, actualización del sitio web y documentación del proyecto. Análogamente, el **Sistema Grow** puede considerarse un actor interno porque ejecuta las operaciones solicitadas: validación de datos, almacenamiento, consulta, filtrado, cálculo de estadísticas y exportación de reportes.

Los actores se resumen en la tabla siguiente:

Actor	Descripción
Usuario final	Persona que utiliza la aplicación para registrar, consultar, filtrar, analizar y exportar sus gastos personales.
Desarrolladora	Persona responsable de construir, probar, corregir, documentar y mantener la aplicación Grow.
Sistema Grow	Aplicación de escritorio que recibe solicitudes del usuario, procesa información, consulta la base de datos y genera resultados.
Base de datos SQLite	Componente interno encargado de almacenar localmente los gastos registrados por el usuario.
Sistema de archivos	Entorno local del equipo donde se guardan los reportes generados en PDF y Excel.
Docente o evaluador	Actor externo que revisa el funcionamiento, documentación, pruebas y cumplimiento de los objetivos del proyecto.

8.2 Casos de uso

Representan las actividades principales realizadas por los actores dentro del sistema. En Grow, los casos de uso están relacionados de manera directa con las funciones implementadas en la aplicación.

La tabla siguiente permite visualizar *grosso modo* dichos casos y sus correspondientes actores:

Código	Caso de uso	Actor principal	Descripción
CU-01	Registrar gasto	Usuario final	El usuario ingresa monto, categoría, descripción y fecha para registrar un nuevo gasto.
CU-02	Validar datos	Sistema Grow	El sistema verifica que los campos requeridos sean correctos antes de guardar el gasto.
CU-03	Guardar gasto	Sistema Grow / SQLite	El sistema almacena el gasto en la base de datos local SQLite.
CU-04	Consultar gastos	Usuario final	El usuario visualiza la lista de gastos registrados en la pantalla principal.
CU-05	Filtrar gastos por categoría	Usuario final	El usuario selecciona una categoría para visualizar únicamente los gastos correspondientes.
CU-06	Eliminar gasto	Usuario final	El usuario elimina un gasto registrado que ya no desea conservar.
CU-07	Consultar estadísticas	Usuario final	El usuario accede a la pantalla de estadísticas para revisar total, promedio y gastos por categoría.
CU-08	Generar reporte PDF	Usuario final	El usuario solicita la exportación de los gastos en formato PDF.
CU-09	Generar reporte Excel	Usuario final	El usuario solicita la exportación de los gastos en formato Excel.
CU-10	Ejecutar pruebas	Desarrolladora	La desarrolladora ejecuta pruebas unitarias, pruebas de widgets y pruebas de rendimiento.
CU-11	Documentar proyecto	Desarrolladora	La desarrolladora elabora el documento técnico y actualiza el sitio web de documentación.

A continuación, se describen algunos de los casos de uso principales de **Grow** en mayor detalle:

CU-01 Registrar gasto

- **Descripción:** el usuario registra un nuevo gasto personal mediante un formulario.
- **Actor principal:** usuario final.
- **Precondiciones:** la aplicación debe estar abierta y el usuario debe acceder a la opción Registrar gasto.
- **Flujo normal:**
 1. El usuario presiona el botón **Registrar gasto**.
 2. El sistema muestra el formulario de registro.
 3. El usuario ingresa el monto, categoría, descripción y fecha.
 4. El usuario presiona el botón **Guardar gasto**.
 5. El sistema valida la información ingresada.
 6. El sistema guarda el gasto en SQLite.
 7. El sistema regresa a la pantalla principal y actualiza la lista de gastos.
- **Flujo alternativo:** si el monto no es válido o algún campo requerido está vacío, el sistema muestra un mensaje de validación.
- **Postcondición:** el gasto queda almacenado en la base de datos local.

CU-04 Consultar gastos

- **Descripción:** el usuario consulta los gastos registrados previamente.
- **Actor principal:** usuario final.
- **Precondiciones:** deben existir gastos almacenados en la base de datos o la lista debe mostrarse vacía si aún no hay registros.
- **Flujo normal:**
 1. El usuario abre la aplicación.
 2. El sistema consulta la base de datos SQLite.
 3. El sistema obtiene los gastos registrados.
 4. La pantalla principal muestra la lista de gastos.
- **Flujo alternativo:** si no existen gastos, el sistema puede mostrar la lista vacía o un mensaje indicando que no hay registros.
- **Postcondición:** el usuario puede visualizar la información guardada.

CU-05 Filtrar gastos por categoría

- **Descripción:** el usuario selecciona una categoría para visualizar solo los gastos relacionados con ella.
- **Actor principal:** usuario final.
- **Precondiciones:** deben existir gastos registrados con categorías asignadas.
- **Flujo normal:**
 1. El usuario abre el filtro de categoría.
 2. Selecciona una categoría específica.

3. El sistema filtra los gastos registrados.
 4. La pantalla principal muestra únicamente los gastos de la categoría seleccionada.
- **Flujo alternativo:** si no existen gastos en la categoría seleccionada, se muestra la lista vacía.
 - **Postcondición:** la lista de gastos queda filtrada según la categoría elegida.

CU-07 Consultar estadísticas

- **Descripción:** el usuario consulta información estadística basada en los gastos registrados.
- **Actor principal:** usuario final.
- **Precondiciones:** la aplicación debe tener acceso a los registros almacenados en SQLite.
- **Flujo normal:**
 1. El usuario presiona el botón **Estadísticas**.
 2. El sistema consulta los gastos registrados.
 3. El sistema calcula total, cantidad de gastos, promedio y gastos por categoría.
 4. El sistema muestra la información estadística y los gráficos correspondientes.
- **Flujo alternativo:** si no hay gastos registrados, el sistema muestra valores en cero o una vista sin datos.
- **Postcondición:** el usuario visualiza un resumen estadístico de sus gastos.

CU-08 Generar reporte PDF

- **Descripción:** el usuario genera un reporte en formato PDF con la información de sus gastos.
- **Actor principal:** usuario final.
- **Precondiciones:** la aplicación debe tener gastos registrados o una lista disponible para exportar.
- **Flujo normal:**
 1. El usuario presiona el botón **Exportar PDF**.
 2. El sistema obtiene los gastos registrados.
 3. El servicio de exportación crea el documento PDF.
 4. El sistema guarda el archivo en el equipo del usuario.
 5. El sistema informa que el reporte fue generado.
- **Flujo alternativo:** si ocurre un error al generar el archivo, el sistema debe evitar el cierre inesperado y mostrar un aviso de error.
- **Postcondición:** el usuario cuenta con un archivo PDF generado localmente.

CU-09 Generar reporte Excel

- **Descripción:** el usuario genera un reporte en formato Excel con la información de sus gastos.
- **Actor principal:** usuario final.
- **Precondiciones:** la aplicación debe tener acceso a la información registrada.
- **Flujo normal:**
 1. El usuario presiona el botón **Exportar Excel**.
 2. El sistema obtiene la lista de gastos.
 3. El servicio de exportación crea el archivo Excel.
 4. El sistema guarda el archivo en el equipo del usuario.
 5. El sistema informa que el reporte fue generado.
- **Flujo alternativo:** si ocurre un error en la creación del archivo, el sistema debe mostrar un aviso y conservar la estabilidad de la aplicación.
- **Postcondición:** el usuario cuenta con un archivo Excel generado localmente.

8.3 Procesos principales

Los procesos principales corresponden a las funciones esenciales que permiten cumplir el objetivo de la aplicación.

Proceso 1: Registro de gastos

Inicia cuando el usuario accede al formulario de registro; el sistema recibe los datos ingresados, valida que el monto sea correcto y que la descripción no esté vacía. Si los datos son válidos, se crea un objeto de gasto y se almacena en SQLite.

Entradas	Proceso	Salida
Monto Categoría Descripción Fecha	Validar datos → Crear gasto → Guardar en SQLite → Actualizar lista principal	Gasto registrado correctamente

Proceso 2: Consulta de gastos

Permite mostrar los gastos almacenados en la base de datos. Al abrir la pantalla principal, el sistema consulta SQLite y obtiene los registros existentes, ordenándolos para su visualización.

Entradas	Proceso	Salida
Solicitud de consulta	Consultar SQLite → Obtener registros → Mostrar lista de gastos	Lista de gastos registrados

Proceso 3: Filtrado por categoría

Permite mostrar únicamente los gastos que corresponden a una categoría seleccionada por el usuario. El sistema toma la lista de gastos y aplica el filtro correspondiente.

Entradas	Proceso	Salida
Categoría seleccionada Lista de gastos registrados	Seleccionar categoría → Filtrar gastos → Actualizar pantalla principal	Lista filtrada por categoría

Proceso 4: Cálculo de estadísticas

Permite analizar los gastos registrados. El sistema obtiene los datos almacenados y calcula información relevante, como total de gastos, cantidad de registros, promedio y gastos por categoría.

Entradas	Proceso	Salida
Lista de gastos registrados	Consultar gastos → Calcular total → Calcular promedio → Agrupar por categoría → Mostrar estadísticas	Resumen estadístico de gastos

Proceso 5: Exportación de reporte PDF

Permite generar un documento con la información de los gastos. El sistema toma los registros almacenados, los organiza en formato de reporte y crea un archivo PDF en el equipo del usuario.

Entradas	Proceso	Salida
Lista de gastos registrados	Obtener gastos → Crear documento PDF → Agregar información → Guardar archivo	Reporte PDF generado

Proceso 6: Exportación de reporte Excel

Permite generar una hoja de cálculo con los gastos registrados. El sistema estructura los datos en filas y columnas, agrega encabezados y guarda el archivo localmente.

Entradas	Proceso	Salida
Lista de gastos registrados	Obtener gastos → Crear archivo Excel → Insertar datos → Guardar archivo	Reporte Excel generado

Proceso 7: Eliminación de gastos

Permite borrar un gasto específico de la base de datos. El usuario selecciona el gasto que desea eliminar y el sistema ejecuta la operación correspondiente en SQLite.

Entradas	Proceso	Salida
Identificador del gasto	Seleccionar gasto → Confirmar eliminación → Eliminar en SQLite → Actualizar lista	Gasto eliminado

Proceso 8: Validación y pruebas del sistema

Corresponde a la verificación del funcionamiento de Grow; la desarrolladora ejecuta pruebas unitarias, pruebas de widgets y pruebas de rendimiento para comprobar que el sistema responda correctamente.

Entradas	Proceso	Salida
Código fuente Casos de prueba Escenarios de uso	Ejecutar Flutter Test → Revisar resultados → Ejecutar DevTools → Documentar evidencia	Resultados de pruebas y evidencias de validación

En la tabla de a continuación se muestra un resumen de los procesos descritos:

Proceso	Actor principal	Componente relacionado	Resultado
Registro de gastos	Usuario final	AddExpenseScreen / DatabaseHelper	Gasto guardado
Consulta de gastos	Usuario final	HomeScreen / DatabaseHelper	Lista de gastos
Filtrado por categoría	Usuario final	HomeScreen	Lista filtrada
Cálculo de estadísticas	Usuario final	StatisticsScreen / StatisticsService	Resumen estadístico
Exportación PDF	Usuario final	PdfExportService	Archivo PDF
Exportación Excel	Usuario final	ExcelExportService	Archivo Excel
Eliminación de gastos	Usuario final	HomeScreen / DatabaseHelper	Gasto eliminado
Validación del sistema	Desarrolladora	Flutter Test / DevTools / ALD	Evidencias de calidad

9 Interfaces del sistema

Las interfaces del sistema **Grow** representan los medios visuales mediante los cuales el usuario interactúa con la aplicación. Al tratarse de una aplicación de escritorio, las interfaces se presentan como pantallas desarrolladas con **Flutter Desktop**, diseñadas para facilitar el registro, consulta, análisis y exportación de gastos personales.

En Grow, la interfaz gráfica cumple una función fundamental, ya que permite que el usuario realice las operaciones principales sin interactuar directamente con la base de datos ni con el código del sistema. Por medio de botones, formularios, listas, filtros y gráficos, el usuario puede administrar su información financiera de manera sencilla.

Las interfaces principales del sistema son:

- Pantalla principal
- Pantalla de registro de gastos
- Pantalla de estadísticas
- Exportación de reportes

9.1 Pantalla principal

Es la primera interfaz que observa el usuario al abrir la aplicación Grow; su función consiste en presentar un resumen general de los gastos registrados y ofrecer acceso a las funciones principales del sistema. Funciona como el centro de navegación de la aplicación. Desde ella, el usuario puede registrar nuevos gastos, consultar estadísticas, exportar reportes, aplicar filtros por categoría y revisar la lista de gastos almacenados.

Los elementos principales de la pantalla principal son:

- Encabezado con el logo de Grow
- Texto descriptivo de la aplicación
- Total mensual de gastos
- Botón Registrar gasto
- Botón Estadísticas
- Botón Exportar PDF
- Botón Exportar Excel
- Filtro por categoría
- Lista de gastos registrados
- Opción para eliminar gastos

La pantalla principal consulta la información almacenada en la base de datos SQLite y la muestra al usuario de forma organizada. Cada gasto registrado aparece con sus datos principales: monto, categoría, descripción y fecha. Además, el filtro por categoría permite que el usuario visualice únicamente los gastos correspondientes a una categoría específica. Esta función mejora la consulta de información y permite analizar de manera más rápida los gastos según su tipo.

Desde el punto de vista funcional, la pantalla principal se relaciona con los siguientes componentes:

Componente	Función
HomeScreen	Presenta la pantalla principal de la aplicación.
DatabaseHelper	Consulta los gastos almacenados en SQLite.
Expense	Representa cada gasto mostrado en pantalla.
PdfExportService	Genera reportes PDF desde la pantalla principal.
ExcelExportService	Genera reportes Excel desde la pantalla principal.

9.2 Pantalla de registro de gastos

Le permite al usuario ingresar la información necesaria para guardar un nuevo gasto personal. Esta interfaz se presenta mediante un formulario sencillo y ordenado.

Los campos principales de esta pantalla son:

- Monto
- Categoría
- Descripción
- Fecha

El campo **monto** permite ingresar la cantidad correspondiente al gasto. El sistema valida que el valor sea numérico y mayor que cero. El campo **categoría** permite seleccionar el tipo de gasto, como alimentación, transporte, salud, educación, entretenimiento, servicios u otros. El campo **descripción** permite agregar una breve explicación del gasto. Finalmente, el campo **fecha** permite indicar cuándo se realizó el gasto.

Los elementos principales de la pantalla son:

- Título Registrar gasto
- Campo para ingresar monto
- Lista desplegable de categoría
- Campo para ingresar descripción
- Selector de fecha

- Botón Guardar gasto
- Mensajes de validación

El flujo de uso de esta pantalla es el siguiente:

1. El usuario abre la pantalla Registrar gasto.
2. Ingresar el monto del gasto.
3. Selecciona la categoría.
4. Escribe una descripción.
5. Selecciona la fecha.
6. Presiona Guardar gasto.
7. El sistema valida los datos.
8. Si los datos son correctos, el gasto se guarda en SQLite.
9. La aplicación regresa a la pantalla principal y actualiza la lista.

Si el usuario deja campos vacíos o ingresa un monto no válido, la aplicación muestra mensajes de validación lo cual permite evitar registros incompletos o incorrectos.

Desde el punto de vista técnico, dicha pantalla se relaciona con los siguientes componentes:

Componente	Función
AddExpenseScreen	Presenta el formulario de registro.
Expense	Crea el objeto de gasto con los datos ingresados.
DatabaseHelper	Guarda el gasto en SQLite.
HomeScreen	Actualiza la lista al regresar del registro.

9.3 Pantalla de estadísticas

Permite al usuario analizar la información de los gastos registrados. Su finalidad es transformar los datos almacenados en información útil para la toma de decisiones personales. Presenta un resumen general de los gastos y permite observar el comportamiento financiero del usuario mediante cálculos y gráficos.

La pantalla de estadísticas muestra información como:

- Total de gastos registrados
- Cantidad de gastos
- Promedio de gastos
- Distribución de gastos por categoría
- Gráfico estadístico de gastos

El proceso de funcionamiento es el siguiente:

1. El usuario presiona el botón Estadísticas.
2. La aplicación obtiene los gastos registrados.
3. El sistema calcula el total de gastos.
4. El sistema calcula la cantidad de registros.
5. El sistema calcula el promedio.
6. El sistema agrupa los gastos por categoría.
7. La pantalla muestra los resultados y el gráfico correspondiente.

Esta interfaz permite que el usuario no solo almacene gastos, sino que también pueda interpretar la información registrada. Por ejemplo, puede identificar en qué categorías gasta más o revisar el promedio de sus gastos.

Los componentes relacionados con esta pantalla son:

Componente	Función
StatisticsScreen	Muestra la pantalla de estadísticas.
StatisticsService	Realiza los cálculos estadísticos.
Expense	Proporciona los datos de cada gasto.
fl_chart	Permite representar gráficamente la información.

9.4 Exportación de reportes

Le permite al usuario generar archivos externos con la información de sus gastos. Esta función se encuentra disponible desde la pantalla principal mediante los botones **Exportar PDF** y **Exportar Excel**. La exportación de reportes permite conservar una copia de los gastos registrados en formatos que pueden consultarse fuera de la aplicación. Esta función es útil para respaldar información, imprimir reportes o analizar los datos en otro programa.

El sistema de Grow permite dos tipos principales de exportación:

- Reporte PDF
- Reporte Excel

Exportación PDF

Genera un documento con la información de los gastos registrados. Este reporte presenta los datos de forma organizada y puede utilizarse como evidencia, resumen o archivo de consulta. El componente principal relacionado con esta función es **PdfExportService**.

El proceso general es el siguiente:

1. El usuario presiona el botón Exportar PDF.
2. El sistema consulta los gastos registrados.
3. El servicio PDF organiza la información.
4. Se genera el archivo PDF.
5. El archivo se guarda localmente en el equipo del usuario.
6. El sistema muestra un mensaje indicando que el reporte fue generado.

Exportación Excel

Genera una hoja de cálculo con los gastos registrados. Este formato permite consultar los datos en columnas y puede ser útil para realizar análisis adicionales. El componente principal relacionado con esta función es **ExcelExportService**.

La exportación de reportes está relacionada con los siguientes componentes:

Componente	Función
HomeScreen	Contiene los botones de exportación.
PdfExportService	Genera el reporte PDF.
ExcelExportService	Genera el reporte Excel.
DatabaseHelper	Proporciona los gastos registrados.
Sistema de archivos	Guarda los archivos generados localmente.

10 Componentes de software

Los componentes de software de **Grow** representan los módulos, clases y servicios que permiten el funcionamiento de la aplicación de escritorio. Estos componentes se encuentran organizados de acuerdo con la arquitectura modular del proyecto, separando la representación de datos, la administración de la base de datos, la lógica de estadísticas y los servicios de exportación.

Los componentes principales del sistema Grow son:

- Modelo Expense
- DatabaseHelper
- StatisticsService
- Servicios de exportación PDF y Excel

10.1 Modelo Expense

Es el componente encargado de representar un gasto dentro de la aplicación Grow. Este modelo define la estructura de datos que utiliza el sistema para almacenar, consultar y manipular la información de cada gasto registrado por el usuario. Permite organizar la información de manera uniforme dentro de la aplicación. Cuando el usuario registra un gasto, el sistema crea un objeto de este tipo con los datos ingresados en el formulario. Posteriormente, ese objeto se convierte en un mapa de datos para poder almacenarse en SQLite.

De igual manera, cuando la aplicación consulta los gastos guardados en la base de datos, cada registro recuperado se transforma nuevamente en un objeto Expense, lo que facilita su visualización en la interfaz y su procesamiento en otros componentes del sistema.

Cada objeto de tipo Expense contiene los datos principales de un gasto personal:

Atributo	Tipo de dato	Descripción
id	int	Identificador único del gasto. Puede ser nulo antes de guardarse en la base de datos
amount	double	Monto del gasto registrado
category	String	Categoría a la que pertenece el gasto
description	String	Descripción breve del gasto
date	DateTime	Fecha en la que se realizó el gasto

Las funciones principales del modelo Expense son:

- ✓ Representar un gasto dentro de la aplicación
- ✓ Definir los atributos básicos de cada registro
- ✓ Convertir un objeto Expense en un mapa para guardarlo en SQLite
- ✓ Convertir un mapa obtenido desde SQLite en un objeto Expense
- ✓ Facilitar el intercambio de información entre la base de datos, las pantallas y los servicios

10.2 DatabaseHelper

Es el encargado de administrar la base de datos local de Grow; su función principal es establecer la conexión con SQLite y ejecutar las operaciones necesarias para guardar, consultar, actualizar o eliminar gastos y forma parte de esa estructura porque conecta la lógica de la aplicación con el almacenamiento local. La base de datos utilizada por Grow se denomina **grow.db**. Dentro de esta base de datos se crea la tabla **expenses**.

Así, la tabla **expenses** contiene los siguientes campos:

Campo	Tipo	Descripción
id	INTEGER	Identificador único autoincremental.
amount	REAL	Monto del gasto.
category	TEXT	Categoría del gasto.
description	TEXT	Descripción del gasto.
date	TEXT	Fecha almacenada como texto.

Este componente permite que Grow funcione como una aplicación local. Gracias a DatabaseHelper, los gastos registrados permanecen guardados en el equipo del usuario y pueden recuperarse cuando la aplicación se abre nuevamente. DatabaseHelper se comunica principalmente con las pantallas de la aplicación. Por ejemplo, la pantalla de registro utiliza este componente para guardar nuevos gastos, mientras que la pantalla principal lo utiliza para consultar la lista de gastos almacenados.

Las responsabilidades principales de **DatabaseHelper** son:

- ✓ Inicializar SQLite para escritorio
- ✓ Crear o abrir la base de datos grow.db
- ✓ Crear la tabla expenses

- ✓ Insertar nuevos gastos
- ✓ Consultar todos los gastos registrados
- ✓ Ordenar los gastos por fecha
- ✓ Eliminar gastos
- ✓ Actualizar gastos cuando sea necesario
- ✓ Cerrar la conexión con la base de datos

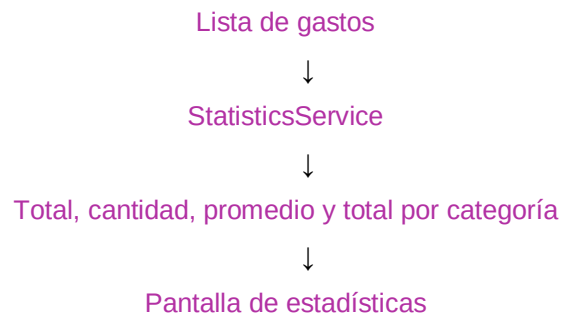
10.3 StatisticsService

Contiene la lógica necesaria para calcular información estadística a partir de los gastos registrados; su función es procesar una lista de objetos Expense y obtener resultados útiles para el usuario. Este servicio permite separar los cálculos estadísticos de la interfaz gráfica. Esta separación mejora la organización del código, facilita el mantenimiento y permite realizar pruebas unitarias sobre la lógica del sistema sin depender directamente de la pantalla.

Las funciones principales de **StatisticsService** son:

Función	Descripción
getTotal	Calcula la suma total de los gastos registrados.
getExpenseCount	Cuenta la cantidad de gastos existentes.
getAverage	Calcula el promedio de los gastos.
getTotalByCategory	Agrupar los gastos por categoría y calcular el total de cada una.

El funcionamiento general del servicio puede representarse de la siguiente manera:



Este componente se utiliza principalmente en la pantalla de estadísticas, donde los datos calculados se presentan al usuario mediante tarjetas, textos y gráficos.

StatisticsService también fue utilizado dentro de las pruebas unitarias del

proyecto. Esto permitió verificar que los cálculos de total, promedio, cantidad de gastos y agrupación por categoría funcionaran correctamente.

10.4 Servicios de exportación PDF y Excel

Permiten generar archivos externos con la información registrada en Grow. Estos componentes amplían la utilidad de la aplicación porque permiten conservar, compartir o revisar los gastos fuera del sistema. Se activan desde la pantalla principal mediante los botones **Exportar PDF** y **Exportar Excel**.

En Grow existen dos servicios principales de exportación:

- PdfExportService
- ExcelExportService

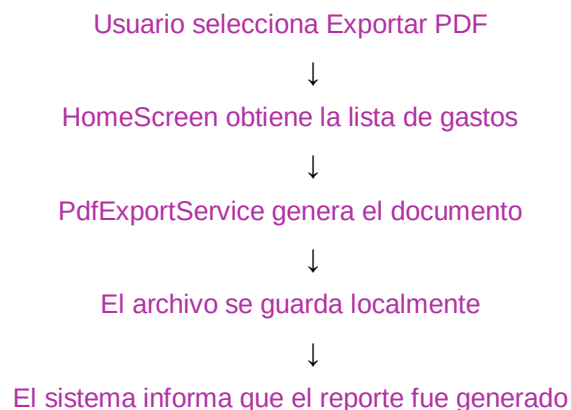
PdfExportService

Este componente se encarga de generar reportes en formato PDF; este servicio toma la lista de gastos registrados y construye un documento organizado con la información correspondiente. El reporte PDF es útil porque permite consultar los gastos en un formato fijo y fácil de imprimir o conservar como evidencia.

Sus responsabilidades principales son:

- ✓ Recibir la lista de gastos
- ✓ Crear un documento PDF
- ✓ Agregar encabezado del reporte
- ✓ Insertar los gastos registrados
- ✓ Calcular o mostrar información de resumen
- ✓ Guardar el archivo PDF en el equipo del usuario

El flujo general de exportación PDF es:



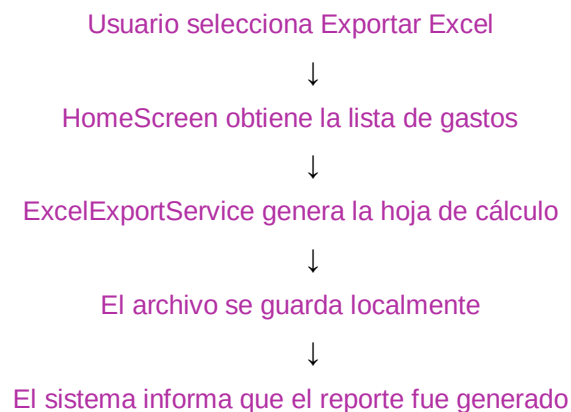
ExcelExportService

Se encarga de generar reportes en formato Excel; este servicio convierte los gastos registrados en una hoja de cálculo organizada por filas y columnas. El reporte Excel es útil porque permite revisar los datos en una hoja de cálculo y realizar análisis adicionales si el usuario lo desea.

Sus responsabilidades principales son:

- ✓ Crear un archivo Excel
- ✓ Agregar encabezados de columnas
- ✓ Insertar monto, categoría, descripción y fecha de cada gasto
- ✓ Agregar información de totales cuando sea necesario
- ✓ Guardar el archivo generado en el equipo del usuario

El flujo general de exportación Excel es:



Relación de los servicios de exportación con otros componentes

Componente	Relación con la exportación
HomeScreen	Contiene los botones para exportar reportes.
Expense	Proporciona la estructura de cada gasto exportado.
DatabaseHelper	Permite consultar los gastos almacenados.
PdfExportService	Genera el archivo PDF.
ExcelExportService	Genera el archivo Excel.
Sistema de archivos	Guarda los reportes generados en el equipo del usuario.

11 Gestión de riesgos

La gestión de riesgos del proyecto **Grow** tiene como finalidad identificar, analizar y prevenir posibles situaciones que puedan afectar el funcionamiento de la aplicación, la integridad de los datos, la generación de reportes o la estabilidad general del sistema. En este proyecto, los riesgos se analizaron desde una perspectiva técnica y de datos. Además, se elaboraron **árboles de fallos** para estudiar eventos críticos que podrían afectar la experiencia del usuario o la confiabilidad de la aplicación.

La estrategia aplicada en Grow es principalmente **preventiva**, ya que busca identificar los posibles problemas antes de que ocurran.

11.1 Riesgos técnicos

Son aquellos que se relacionan con el funcionamiento interno de la aplicación, las herramientas de desarrollo, las dependencias utilizadas, la base de datos, la interfaz gráfica y el ambiente de ejecución. En Grow, estos riesgos pueden afectar directamente la capacidad del sistema para registrar gastos, mostrar información, calcular estadísticas o generar reportes.

La tabla siguiente muestra las acciones preventivas tomadas para cada uno de los principales errores:

Riesgo técnico	Descripción	Impacto	Acción preventiva
Error en la configuración de Flutter Desktop	La aplicación puede no ejecutarse correctamente si Flutter no está configurado para Windows.	Alto	Verificar el ambiente con flutter doctor y probar la aplicación en modo escritorio.
Fallo en dependencias del proyecto	Algún paquete puede causar incompatibilidades o errores de compilación.	Medio	Revisar pubspec.yaml, ejecutar flutter pub get y evitar paquetes incompatibles.
Error en SQLite	La base de datos puede no inicializarse o la tabla puede no crearse correctamente.	Alto	Validar la creación de grow.db y la tabla expenses.
Error en la interfaz	Alguna pantalla puede presentar errores visuales, desbordamientos o elementos mal organizados.	Medio	Realizar pruebas manuales y pruebas de widgets.
Error en navegación	La aplicación puede no regresar correctamente a la pantalla principal después de registrar un gasto.	Medio	Verificar el uso de Navigator.pop y la recarga de datos.
Fallo en generación	El reporte puede no crearse	Alto	Probar la exportación PDF

Riesgo técnico	Descripción	Impacto	Acción preventiva
de PDF	correctamente o guardarse en una ruta incorrecta.		con distintos registros y validar el archivo generado.
Fallo en generación de Excel	El archivo Excel puede no generarse, quedar vacío o presentar datos incompletos.	Alto	Probar la exportación Excel y revisar que contenga encabezados, gastos y totales.
Cierre inesperado de la aplicación	La aplicación puede cerrarse por errores no controlados en dependencias, exportaciones o base de datos.	Alto	Ejecutar pruebas, revisar errores en consola y controlar excepciones.
Bajo rendimiento	La aplicación puede responder lentamente al cargar datos, generar estadísticas o exportar reportes.	Medio	Evaluar el rendimiento con Flutter DevTools.

11.2 Riesgos de datos

Están relacionados con la pérdida, corrupción, duplicación, inconsistencia o mal almacenamiento de la información registrada por el usuario. En Grow, estos riesgos son importantes porque la finalidad principal de la aplicación es conservar y consultar gastos personales. Cabe mencionar que estos riesgos se adaptan al contexto de una aplicación local de escritorio. Cabe señalar, asimismo, que en esta primera versión, Grow no usa almacenamiento en la nube ni sincronización externa y, por lo tanto, el resguardo de los datos depende del equipo donde se ejecuta la aplicación. Esta característica es adecuada para el alcance académico del proyecto, pero representa una limitación importante si se desea convertir Grow en una aplicación de uso más amplio.

La tabla de a continuación muestra los principales riesgos en este proyecto:

Riesgo de datos	Descripción	Impacto	Acción preventiva
Pérdida de gastos registrados	Los gastos pueden perderse si la base de datos se elimina, se daña o no se guarda correctamente.	Alto	Verificar que SQLite guarde correctamente los registros.
Datos incompletos	El usuario podría intentar guardar un gasto sin monto, descripción o categoría válida.	Alto	Implementar validaciones en el formulario de registro.
Monto inválido	El usuario podría ingresar texto, cero o números negativos como monto.	Medio	Validar que el monto sea numérico y mayor que cero.
Fecha incorrecta	El gasto podría guardarse con una fecha no deseada.	Medio	Usar selector de fecha y mostrar la fecha seleccionada antes de guardar.

Riesgo de datos	Descripción	Impacto	Acción preventiva
Inconsistencia entre lista y base de datos	Un gasto puede guardarse en SQLite pero no aparecer en la pantalla principal.	Alto	Recargar la lista después de guardar, eliminar o actualizar registros.
Error al convertir datos	Puede fallar la conversión entre objetos Expense y mapas de SQLite.	Alto	Revisar los métodos toMap() y fromMap().
Reportes incompletos	Los reportes PDF o Excel pueden generarse sin todos los gastos.	Medio	Usar la lista actualizada de gastos al momento de exportar.
Eliminación accidental	El usuario puede borrar un gasto que todavía necesitaba.	Medio	Documentar el comportamiento de eliminación y considerar confirmación en futuras versiones.
Dependencia del equipo local	Los datos se almacenan en la computadora donde se usa Grow.	Medio	Recomendar respaldos manuales o considerar respaldo en futuras versiones.

11.3 Árboles de fallos

En Grow, los árboles de fallos se usaron como herramienta preventiva para analizar fallos críticos que podrían afectar el funcionamiento del sistema. El análisis fue realizado con la herramienta **ALD Free Web Fault Tree Analysis Tool**. Es importante señalar que en este proyecto los árboles se utilizaron de manera únicamente **cualitativa**, es decir, para identificar causas y relaciones lógicas, pero no para realizar cálculos probabilísticos. Esto se debe a que el proyecto no cuenta con datos históricos reales sobre frecuencia de fallos.

Los eventos principales analizados fueron:

- El gasto válido no se registra o no aparece en la lista.
- El reporte PDF no se genera correctamente.
- La aplicación se cierra inesperadamente.

Árbol de fallos 1: El gasto válido no se registra o no aparece en la lista

Este árbol analiza las causas por las cuales un gasto que fue ingresado correctamente por el usuario podría no guardarse o no mostrarse en la pantalla principal. Este análisis permitió identificar que las causas más críticas se relacionan con el almacenamiento en SQLite y la actualización de la pantalla principal después del registro.

Las causas principales consideradas fueron:

Causa principal	Descripción
Error en el formulario de registro	Puede existir un problema en los campos de monto, categoría, descripción o fecha
Error en la validación de datos	El sistema puede rechazar datos válidos o aceptar datos incompletos
Error al guardar en SQLite	La base de datos puede no inicializarse o fallar al insertar el registro
Error al consultar los gastos guardados	La consulta puede no recuperar correctamente los registros almacenados
Error al actualizar la pantalla principal	La interfaz puede no recargar la lista después de guardar el gasto
Error en el ambiente de prueba	El problema puede deberse a dependencias, configuración o datos insuficientes

Árbol de fallos 2: El reporte PDF no se genera correctamente

Este árbol analiza los posibles motivos por los cuales la exportación de reportes en formato PDF puede fallar. Este análisis permitió identificar que la generación de reportes depende de tres elementos principales: la correcta obtención de datos, el funcionamiento del servicio PDF y el acceso a una ruta válida de almacenamiento.

Las causas principales consideradas fueron:

Causa principal	Descripción
Error al obtener los gastos	La lista de gastos puede estar vacía, desactualizada o no recuperarse desde SQLite
Error en el servicio PDF	Puede fallar la construcción del documento o la inserción de datos
Error de formato	El reporte puede generarse con información incompleta, mal ordenada o ilegible
Error en la ruta de guardado	El archivo puede no guardarse si la ruta no existe o no hay permisos
Error de dependencia	La librería utilizada para crear PDF puede presentar incompatibilidades
Error durante la ejecución	La aplicación puede detenerse si no se controla una excepción

Árbol de fallos 3: La aplicación se cierra inesperadamente

Este árbol analiza las causas que podrían provocar que Grow se cierre durante su uso. Este árbol permitió reconocer que el cierre inesperado puede estar relacionado con errores de base de datos, exportación, navegación o dependencias. Por ello, es importante realizar pruebas frecuentes y observar la salida de la terminal durante la ejecución de la aplicación.

Las causas principales consideradas fueron:

Causa principal	Descripción
Error no controlado en la aplicación	Una excepción puede provocar el cierre del sistema
Error en SQLite	Problemas de conexión, consulta o escritura pueden afectar la estabilidad
Error en dependencias	Algún paquete puede generar fallos durante la ejecución
Error en exportación de reportes	La generación de PDF o Excel puede provocar errores si no se controla correctamente
Error en navegación	La transición entre pantallas puede fallar si el contexto no está disponible
Error en ambiente de ejecución	La configuración de Flutter Desktop o Windows puede afectar la ejecución

12 Verificación de calidad

La verificación de calidad del proyecto **Grow** tiene como finalidad comprobar que la aplicación cumpla con los requisitos funcionales y no funcionales establecidos. Esta verificación permite evaluar si el sistema registra gastos correctamente, conserva la información, muestra los datos en la interfaz, calcula estadísticas y genera reportes sin errores críticos. Para esta etapa se utilizaron principalmente **Flutter Test** y **Flutter DevTools**, además de la revisión manual de la aplicación durante su ejecución.

Las pruebas realizadas se enfocaron en validar los componentes principales del sistema:

- Lógica de cálculo estadístico.
- Funcionamiento de formularios.
- Visualización de elementos de interfaz.
- Estabilidad general de la aplicación.
- Rendimiento durante el uso de funciones principales.

12.1 Pruebas unitarias

Estas permiten verificar el funcionamiento de componentes específicos del sistema de manera aislada. En Grow, estas pruebas se aplicaron principalmente al componente **StatisticsService**, debido a que este servicio contiene la lógica necesaria para calcular totales, promedios, cantidad de gastos y agrupación por categoría. El objetivo de estas pruebas fue comprobar que los cálculos realizados por el sistema fueran correctos antes de mostrarlos al usuario en la pantalla de estadísticas.

Estas pruebas fueron importantes porque la pantalla de estadísticas depende directamente de los resultados generados por **StatisticsService**. Si este servicio presenta errores, el usuario podría recibir información incorrecta sobre sus gastos. El archivo principal utilizado para estas pruebas fue **test/statistics_service_test.dart**. El comando utilizado para ejecutar las pruebas fue **flutter test**.

Las funciones evaluadas fueron las siguientes:

Función evaluada	Objetivo de la prueba
getTotal	Verificar que el sistema calcule correctamente la suma total de gastos.
getExpenseCount	Comprobar que el sistema cuente correctamente la cantidad de registros.
getAverage	Validar que el promedio de gastos se calcule correctamente.
getTotalByCategory	Comprobar que los gastos se agrupen correctamente por categoría.

12.2 Pruebas de widgets

Estas permiten verificar que los elementos visuales de la aplicación se muestren correctamente y respondan de acuerdo con el comportamiento esperado. En Grow, estas pruebas se aplicaron a la interfaz de registro de gastos y a elementos generales de la aplicación.

La pantalla de registro de gastos fue una de las más importantes para esta etapa, ya que representa el punto de entrada de información al sistema. Por ello, se verificó que el formulario pudiera mostrarse correctamente y que sus elementos principales estuvieran disponibles para el usuario.

Las pruebas de widgets ayudan a detectar errores en la interfaz antes de que afecten la experiencia del usuario. Por ejemplo, permiten identificar si un botón no aparece, si un campo no se muestra o si una pantalla no se construye correctamente.

Los archivos utilizados fueron:

test/add_expense_screen_test.dart
test/widget_test.dart

Las pruebas de widgets permitieron comprobar aspectos como:

- Visualización de la pantalla de registro
- Presencia de campos del formulario
- Presencia de botones principales
- Funcionamiento básico de elementos de interfaz
- Construcción correcta de widgets principales

12.3 Pruebas de rendimiento

Estas permiten observar el comportamiento de la aplicación durante su ejecución. En Grow, estas pruebas se realizaron con **Flutter DevTools**, ejecutando la aplicación en modo **profile**. El objetivo de esta prueba fue evaluar la estabilidad y fluidez de la aplicación durante el uso de sus funciones principales. El comando utilizado fue **flutter run --profile -d windows**.

Durante la prueba se revisaron acciones como:

- Navegación entre pantallas
- Cambio de filtro por categoría
- Consulta de estadísticas
- Generación de reportes PDF
- Generación de reportes Excel
- Actualización de la lista de gastos

En Flutter DevTools se observó el comportamiento de los frames durante la ejecución. La aplicación mostró principalmente frames en color azul, lo cual indica un comportamiento estable en la mayor parte del uso. También se observaron algunos frames lentos marcados en color rojo o naranja, posiblemente relacionados con operaciones de mayor carga, como la generación de reportes o actualización de información. El promedio observado fue cercano a **55 FPS**, por lo que el rendimiento

se considera aceptable para el alcance académico del proyecto. La aplicación no presentó bloqueos visibles, cierres inesperados ni errores críticos durante la prueba.

12.4 Resultados obtenidos

Los resultados obtenidos durante la verificación de calidad fueron satisfactorios. La ejecución general de pruebas con Flutter Test mostró que todas las pruebas fueron aprobadas correctamente. El resultado general fue: **+15: All tests passed!** Esto significa que se ejecutaron 15 pruebas en total y todas fueron aprobadas.

La distribución de pruebas fue la siguiente:

Archivo de prueba	Tipo de prueba	Cantidad aproximada
test/statistics_service_test.dart	Pruebas unitarias	11 pruebas
test/add_expense_screen_test.dart	Pruebas de widgets	3 pruebas
test/widget_test.dart	Prueba general de widget	1 prueba
Total	Unitarias y widgets	15 pruebas

Los resultados principales fueron:

- Las funciones de StatisticsService calcularon correctamente totales, promedios y agrupaciones por categoría.
- La pantalla de registro de gastos se construyó correctamente.
- Los widgets principales de la aplicación fueron reconocidos durante las pruebas.
- La aplicación se ejecutó correctamente en modo profile.
- Flutter DevTools permitió observar un rendimiento estable.
- No se detectaron cierres inesperados durante la prueba de rendimiento.
- La generación de reportes PDF y Excel funcionó correctamente durante la validación manual.

La verificación de calidad permitió comprobar que **Grow** cumple con sus funciones principales y presenta un comportamiento estable. Las pruebas automatizadas confirmaron el correcto funcionamiento de la lógica y de algunos elementos de interfaz, mientras que las pruebas de rendimiento permitieron evaluar la fluidez general de la aplicación durante su uso.

Elemento verificado	Herramienta utilizada	Resultado
Cálculos estadísticos	Flutter Test	Correcto
Formulario de registro	Flutter Test	Correcto
Widgets principales	Flutter Test	Correcto
Rendimiento general	Flutter DevTools	Aceptable
Exportación PDF	Prueba manual	Correcta
Exportación Excel	Prueba manual	Correcta

Elemento verificado	Herramienta utilizada	Resultado
Estabilidad de ejecución	Flutter DevTools / prueba manual	Correcta

13 Publicación y control de versiones

La publicación y el control de versiones del proyecto **Grow** permiten administrar los cambios realizados durante el desarrollo, conservar un historial del código fuente y presentar públicamente la documentación del sistema. Para ello se utilizaron tres herramientas principales: **Git**, **GitHub** y **GitHub Pages**.

El control de versiones es importante porque permite registrar cada avance significativo del proyecto, identificar modificaciones realizadas en los archivos y conservar versiones estables de la aplicación. Además, facilita el respaldo del código fuente y la organización de los entregables técnicos. En el caso de Grow, el control de versiones se aplicó tanto al código de la aplicación Flutter como a los archivos de documentación web ubicados en la carpeta **docs**.

Las herramientas utilizadas fueron:

- **Git**: control de versiones local
- **GitHub**: repositorio remoto del proyecto
- **GitHub Pages**: publicación del sitio web de documentación

13.1 Git

Fue usado como sistema de control de versiones local para el proyecto Grow. Esta herramienta permitió registrar los cambios realizados en el código fuente, conservar un historial de modificaciones y preparar los archivos para ser enviados al repositorio remoto en GitHub. El proyecto fue desarrollado en una carpeta local de Windows. Desde la terminal integrada de Visual Studio Code se ejecutaron los comandos necesarios para revisar el estado del proyecto, agregar cambios, crear **commits** y enviar los avances al repositorio remoto.

Durante el desarrollo de Grow, Git permitió controlar los cambios en archivos como:

- Archivos Dart de la aplicación
- Pantallas de la interfaz
- Servicios de base de datos
- Servicios de exportación PDF y Excel
- Archivos de pruebas
- Archivos de configuración
- Archivos del sitio web de documentación

Los comandos principales utilizados fueron:

git status

git add .

git commit -m "mensaje del cambio"

git push

El comando **git status** permitió revisar si existían archivos modificados, nuevos archivos o cambios pendientes. El comando **git add .** se utilizó para preparar todos los cambios realizados. Posteriormente, **git commit -m** permitió guardar una versión local con un mensaje descriptivo. Finalmente, **git push** permitió enviar los cambios al repositorio en GitHub.

Beneficios de Git en Grow

Beneficio	Descripción
Historial de cambios	Permite consultar las modificaciones realizadas durante el desarrollo.
Organización del proyecto	Ayuda a registrar avances por etapas.
Respaldo local	Conserva versiones del proyecto en el equipo de desarrollo.
Control de errores	Permite identificar cambios que pudieron generar fallos.
Preparación para GitHub	Facilita el envío del proyecto al repositorio remoto.

13.2 GitHub

Se usó como plataforma de alojamiento remoto para el repositorio del proyecto Grow. A través de GitHub, el código fuente quedó respaldado en línea y disponible para su consulta desde un navegador web. Dentro del repositorio se encuentran los archivos principales de la aplicación, incluyendo la estructura del proyecto Flutter, las dependencias, las pruebas, los recursos gráficos y la documentación web. GitHub permitió respaldar el proyecto y conservarlo fuera del equipo local. Esto es importante porque reduce el riesgo de pérdida de código y facilita la revisión del trabajo desarrollado. Además, funcionó como evidencia del proceso de desarrollo, ya que permite consultar el historial de commits, la fecha de los cambios y los archivos modificados.

El repositorio remoto del proyecto es:

https://github.com/a374279-debug/grow_app

Entre los elementos almacenados en GitHub se encuentran:

- Carpeta lib
- Carpeta test
- Carpeta assets
- Carpeta docs
- Carpeta windows
- Archivo pubspec.yaml
- Archivo README.md

Función	Uso en el proyecto
Repositorio remoto	Almacenamiento del código fuente de Grow.
Historial de commits	Revisión de cambios realizados durante el desarrollo.
Rama principal	Organización del proyecto en la rama main.
Respaldo del proyecto	Conservación del código en una plataforma externa al equipo local.
Publicación web	Uso de GitHub Pages para mostrar la documentación del proyecto.

13.3 GitHub Pages

Se utilizó para publicar el sitio web de documentación del proyecto Grow. Esta herramienta permite convertir una carpeta del repositorio en una página web pública, accesible desde cualquier navegador. El sitio web publicado del proyecto es:

https://a374279-debug.github.io/grow_app/

La documentación web se encuentra dentro de la carpeta **docs**. Dentro de esta carpeta se incluyen los archivos principales del sitio:

```
docs
├── index.html
├── styles.css
├── img
└── logo_grow.png
```

El archivo **index.html** contiene la estructura del sitio web, mientras que **styles.css** define el diseño visual, colores, distribución, tarjetas, tablas y adaptación a pantallas pequeñas. La carpeta **img** contiene el logo utilizado en el encabezado del sitio. El sitio web de Grow presenta información general del proyecto, sus funcionalidades principales, herramientas utilizadas, manual básico de uso, estructura del proyecto, estado actual y conclusión.

14 Manual básico de uso

El presente manual básico de uso describe las pantallas principales de la aplicación **Grow** y explica las acciones que el usuario puede realizar en cada una de ellas. Su finalidad es servir como guía rápida para comprender el funcionamiento general del sistema y utilizar correctamente sus funciones principales.

Grow fue diseñado como una aplicación de escritorio para la gestión de gastos personales. A través de sus pantallas, el usuario puede registrar gastos, consultar la lista de registros, aplicar filtros, revisar estadísticas y exportar reportes en formato PDF y Excel.

14.1 Inicio de la aplicación

Para utilizar Grow, el usuario debe abrir la aplicación en un equipo con sistema operativo Windows. Una vez iniciada, el sistema muestra la pantalla principal, desde la cual se puede acceder a las funciones principales.

El flujo inicial de uso es el siguiente:

1. Abrir la aplicación Grow.
2. Esperar a que cargue la pantalla principal.
3. Revisar el total mensual y la lista de gastos registrados.
4. Elegir la función deseada: registrar gasto, consultar estadísticas, filtrar información o exportar reportes.



Ilustración 5. Logo de Grow

14.2 Pantalla principal

La pantalla principal es la interfaz central de Grow. En ella se muestran los gastos registrados, el total mensual y los botones de acceso a las funciones principales. El usuario puede utilizar esta pantalla para revisar rápidamente sus gastos recientes y acceder a las demás funciones del sistema.

Los elementos que aparecen en esta pantalla son:

Elemento	Función
Logo de Grow	Identifica visualmente la aplicación
Total mensual	Muestra la suma de los gastos registrados durante el mes actual
Botón Registrar gasto	Abre el formulario para agregar un nuevo gasto
Botón Estadísticas	Abre la pantalla de análisis y resumen de gastos
Botón Exportar PDF	Genera un reporte en formato PDF
Botón Exportar Excel	Genera un reporte en formato Excel
Filtro por categoría	Permite mostrar gastos de una categoría específica
Lista de gastos	Presenta los gastos registrados con monto, categoría, descripción y fecha
Opción eliminar	Permite borrar un gasto registrado

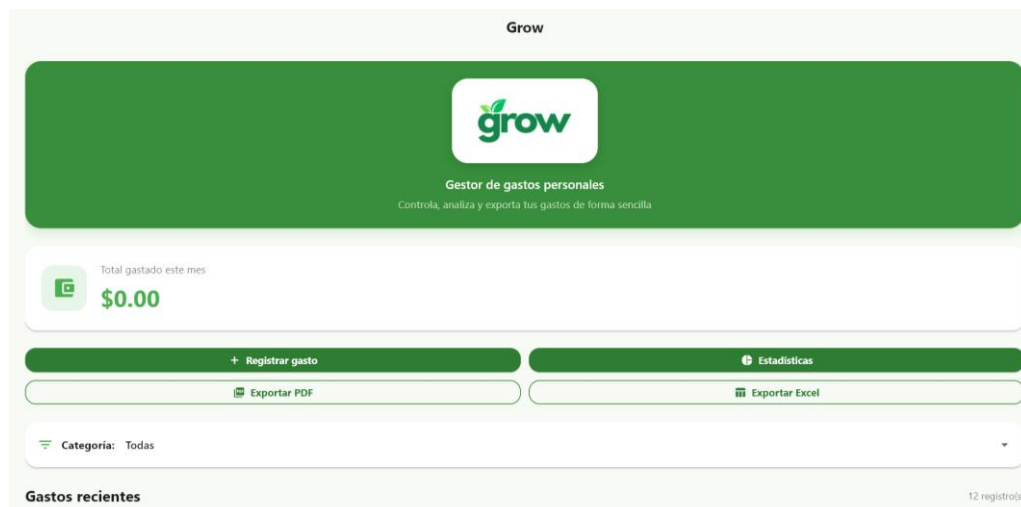


Ilustración 6. Pantalla principal de Grow

14.3 Registro de un nuevo gasto

Para registrar un gasto, el usuario debe presionar el botón **Registrar gasto** desde la pantalla principal. Después, el sistema muestra un formulario donde se deben ingresar los datos correspondientes. Si el usuario deja campos vacíos o ingresa un monto incorrecto, el sistema muestra mensajes de validación. Esto evita que se almacenen datos incompletos o inválidos.

Los campos del formulario son:

Campo	Descripción
Monto	Cantidad de dinero correspondiente al gasto
Categoría	Tipo de gasto, por ejemplo, alimentación, transporte, salud, educación, entretenimiento, servicios u otros
Descripción	Breve explicación del gasto realizado
Fecha	Día en que se realizó el gasto

El procedimiento para registrar un gasto es el siguiente:

1. Presionar el botón Registrar gasto.
2. Ingresar el monto del gasto.
3. Seleccionar una categoría.
4. Escribir una descripción.
5. Seleccionar la fecha correspondiente.
6. Presionar el botón Guardar gasto.
7. Esperar a que la aplicación regrese a la pantalla principal.
8. Verificar que el gasto aparezca en la lista.

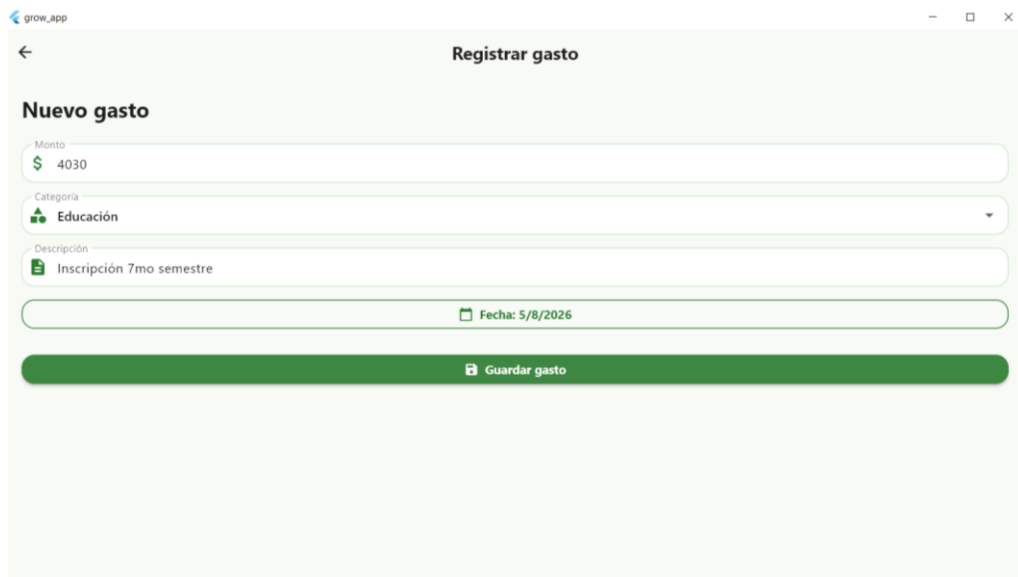


Ilustración 7. Pantalla de registro de gastos

14.4 Consulta de gastos registrados

La consulta de gastos se realiza desde la pantalla principal. Al abrir la aplicación, Grow obtiene los registros guardados en la base de datos SQLite y los muestra en forma de lista. Esta función permite al usuario llevar un control visual de sus gastos personales registrados.

Cada gasto muestra información como:

- Monto
- Categoría
- Descripción
- Fecha

El procedimiento para consultar los gastos es el siguiente:

1. Abrir la aplicación Grow.
2. Revisar la lista de gastos en la pantalla principal.
3. Identificar el gasto deseado por su categoría, descripción, monto o fecha.

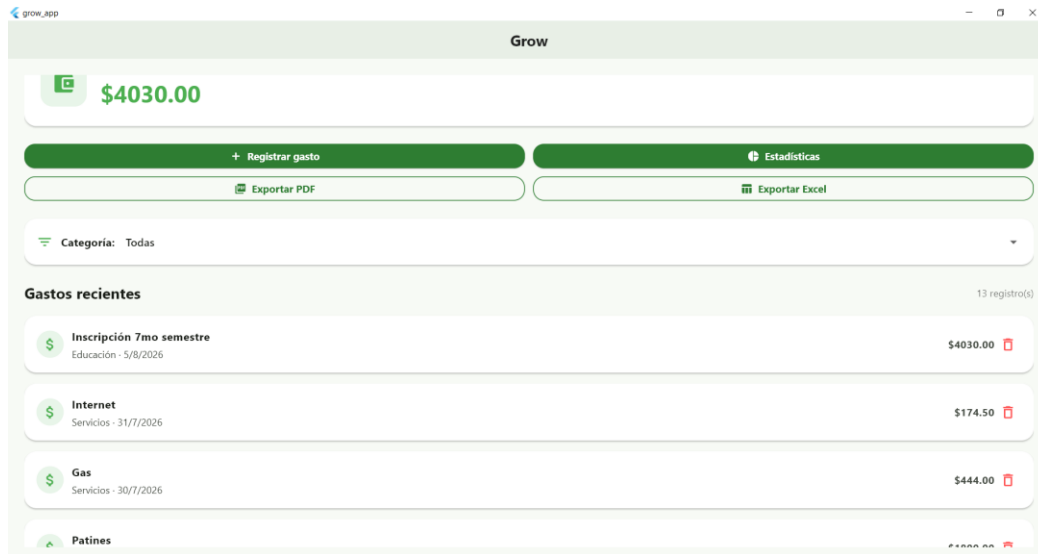


Ilustración 8. Pantalla de lista de gastos registrados

14.5 Filtrado por categoría

Grow permite filtrar los gastos por categoría. Esta función ayuda al usuario a revisar únicamente los registros relacionados con un tipo específico de gasto. Por ejemplo, si el usuario selecciona la categoría **Transporte**, la aplicación mostrará únicamente los gastos registrados dentro de esa categoría. Esta función facilita el análisis de hábitos de consumo y permite identificar en qué áreas se concentra una mayor cantidad de gastos.

El procedimiento para filtrar gastos es el siguiente:

1. Ubicar el filtro de categoría en la pantalla principal.
2. Seleccionar la categoría deseada.
3. Revisar la lista actualizada.
4. Para volver a ver todos los gastos, seleccionar la opción Todas.

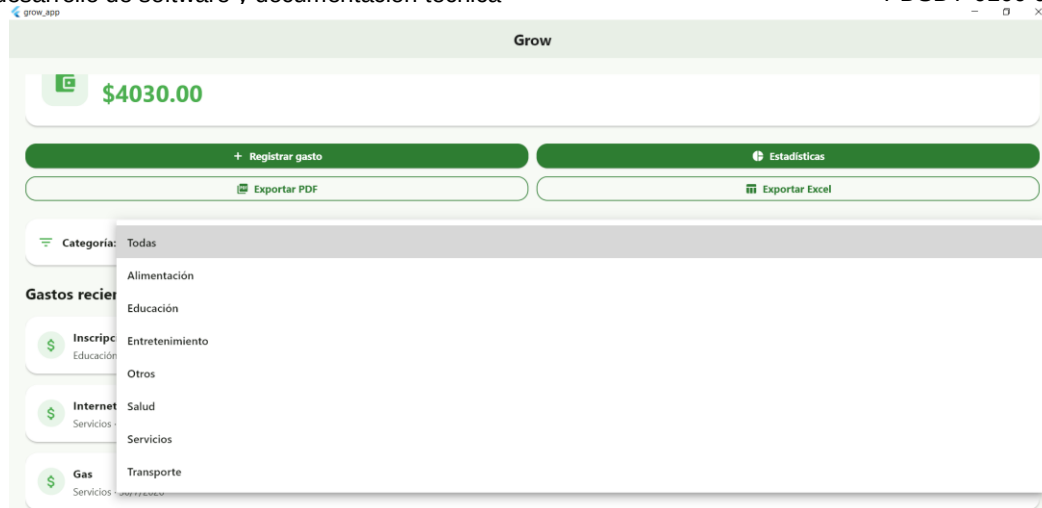


Ilustración 9. Pantalla de filtro de gastos por categoría

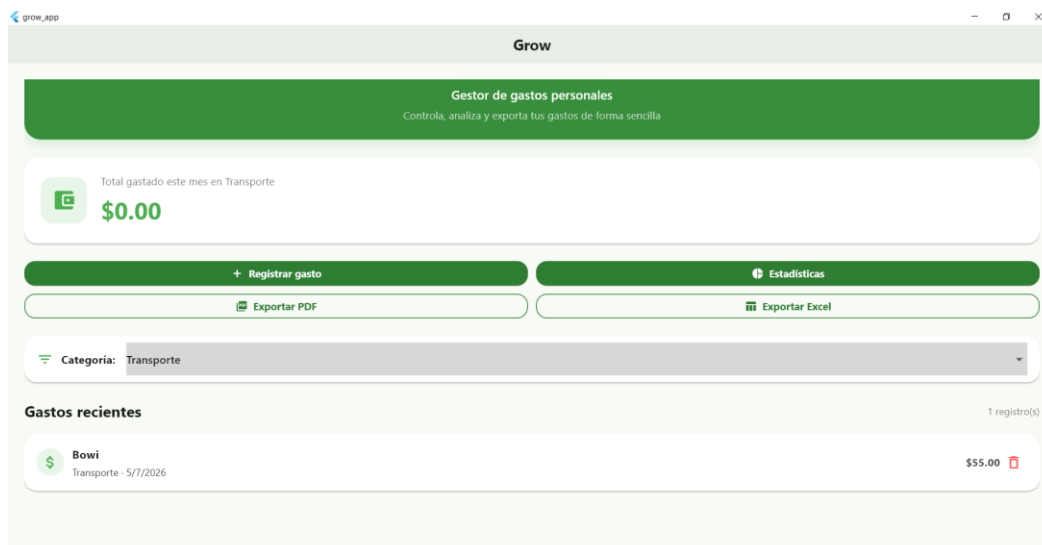


Ilustración 10. Pantalla de lista de gastos filtrados por categoría "Transporte"

14.6 Eliminación de gastos

La pantalla principal también permite eliminar gastos registrados. Esta función es útil cuando el usuario capturó un gasto por error o desea retirar un registro de la lista. Cuando se elimina un gasto, la aplicación actualiza la lista y el total mensual. Esta acción modifica la información almacenada en la base de datos local.

El procedimiento para eliminar un gasto es el siguiente:

1. Buscar el gasto en la lista principal.
2. Presionar el botón o ícono de eliminar.
3. Esperar a que el sistema elimine el registro.
4. Verificar que el gasto ya no aparezca en la lista.

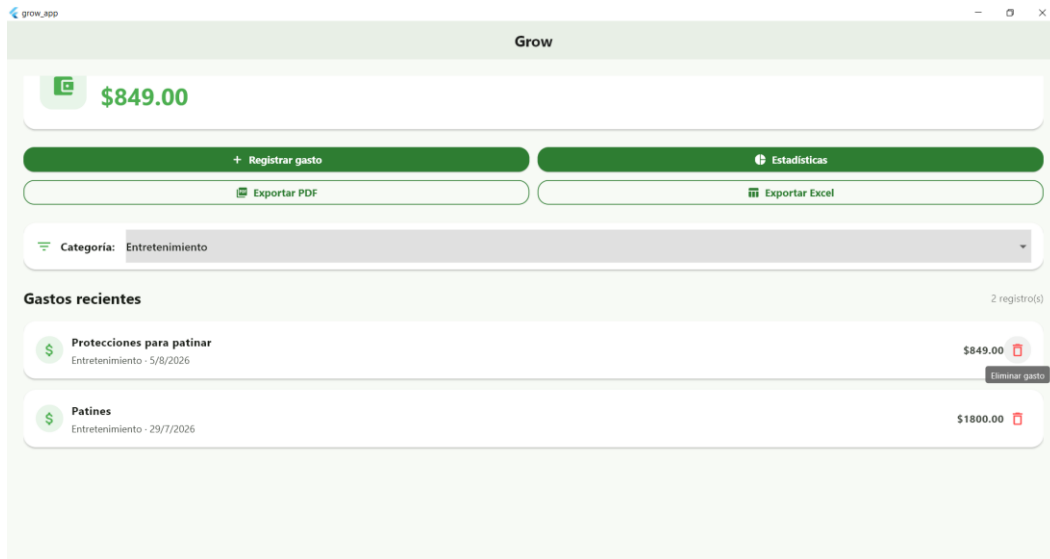


Ilustración 11. Pantalla de botón de eliminación de gasto

14.7 Pantalla de estadísticas

La pantalla de estadísticas permite consultar un resumen de los gastos registrados. Para acceder a esta función, el usuario debe presionar el botón **Estadísticas** desde la pantalla principal. Esta función ayuda al usuario a interpretar sus gastos de manera más clara pues convierte los registros almacenados en información resumida y visual.

Esta pantalla muestra información como:

Elemento	Descripción
Total de gastos	Suma total de los gastos registrados.
Cantidad de gastos	Número total de registros almacenados.
Promedio	Promedio general de los gastos.
Gastos por categoría	Distribución de los gastos según su categoría.
Gráfico	Representación visual de la información por categoría.

El procedimiento para consultar estadísticas es el siguiente:

1. Presionar el botón Estadísticas.
2. Esperar a que la aplicación cargue los datos.
3. Revisar el total, cantidad y promedio de gastos.
4. Observar la distribución por categoría.
5. Analizar el gráfico mostrado en pantalla.
6. Regresar a la pantalla principal cuando sea necesario.

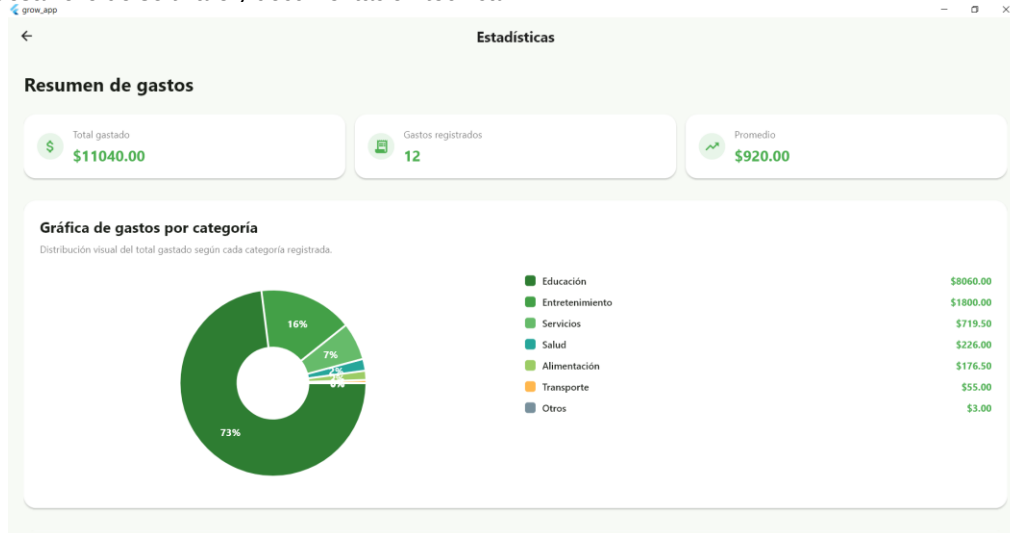


Ilustración 12. Pantalla de gráfico de gastos

14.8 Exportación de reportes PDF

Grow permite generar un reporte en formato PDF con la información de los gastos registrados. Esta función está disponible desde la pantalla principal mediante el botón **Exportar PDF**. Este último puede usarse como evidencia, resumen de gastos o archivo de consulta. Su formato permite conservar la información de manera ordenada y fácil de visualizar.

El procedimiento para exportar un reporte PDF es el siguiente:

1. Abrir la pantalla principal.
2. Verificar que existan gastos registrados.
3. Presionar el botón Exportar PDF.
4. Esperar a que el sistema genere el archivo.
5. Revisar el mensaje de confirmación.
6. Abrir el archivo PDF desde la carpeta donde fue guardado.

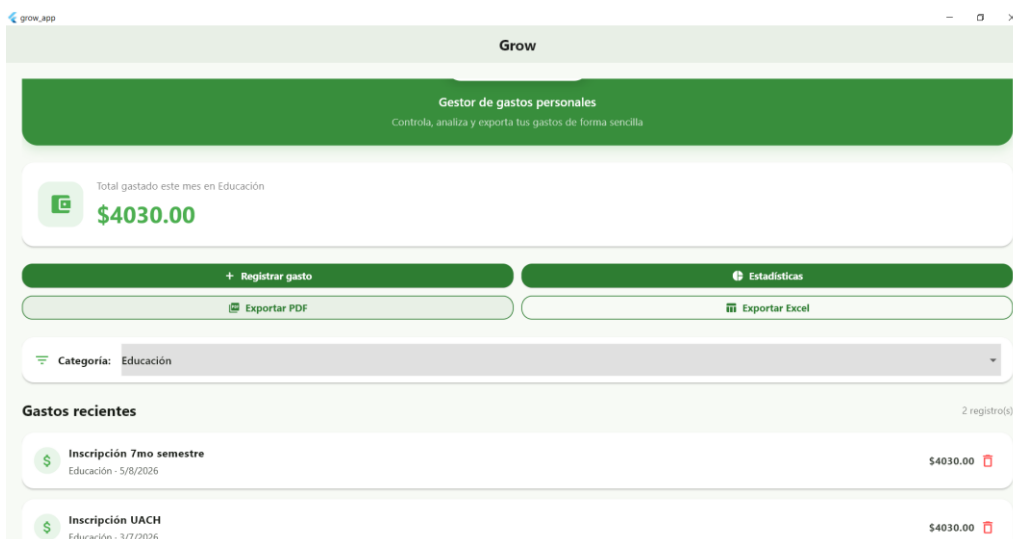


Ilustración 13. Pantalla de botón "Exportar PDF"

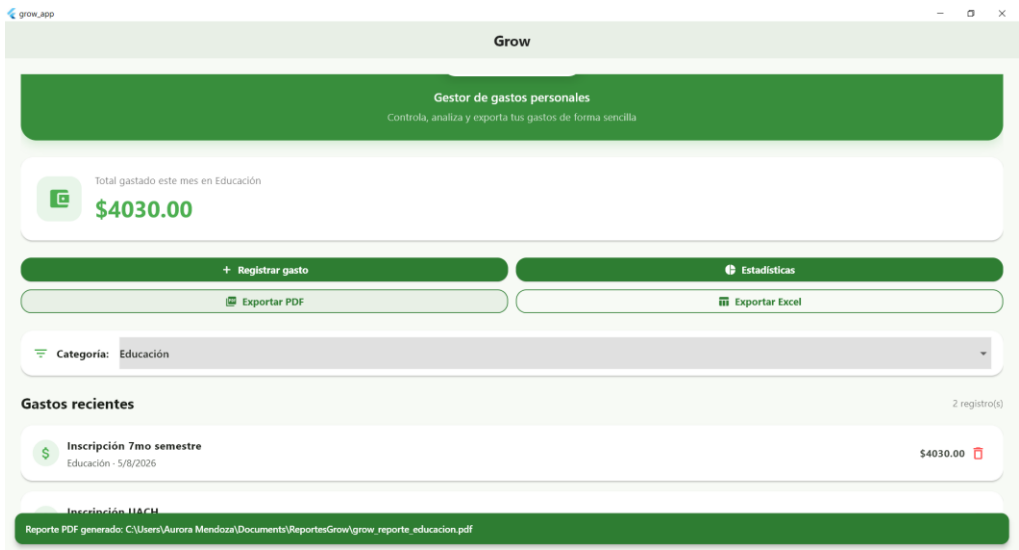


Ilustración 14. Pantalla de mensaje de exportación de PDF

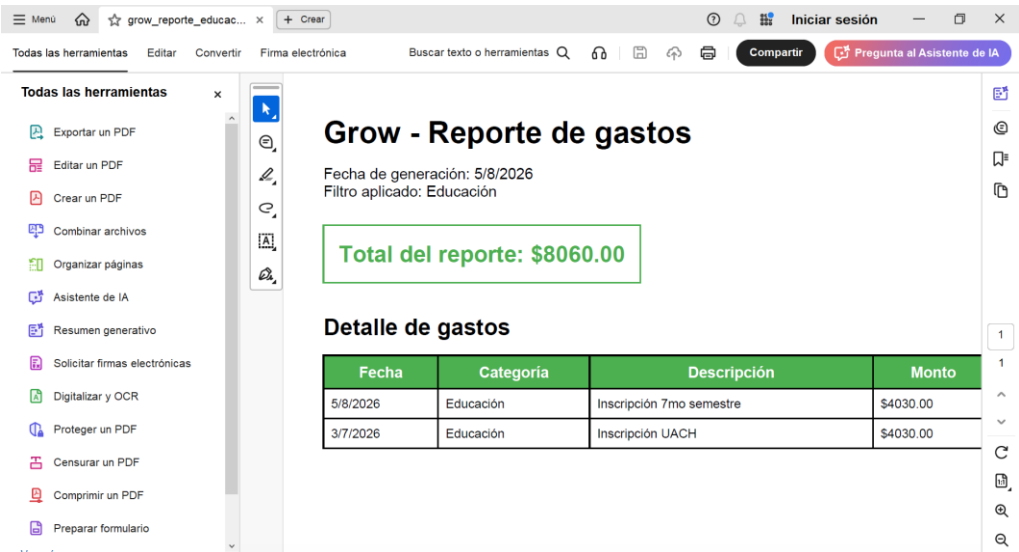


Ilustración 15. Pantalla del archivo PDF exportado

14.9 Exportación de reportes Excel

La aplicación también permite exportar los gastos en formato Excel. Esta función es útil cuando el usuario desea consultar los datos en una hoja de cálculo o realizar análisis adicionales. El archivo Excel organiza los datos en filas y columnas, lo que permite revisar la información de manera estructurada.

El procedimiento para exportar un reporte Excel es el siguiente:

1. Abrir la pantalla principal.
2. Verificar que existan gastos registrados.
3. Presionar el botón Exportar Excel.
4. Esperar a que el sistema genere el archivo.
5. Revisar el mensaje de confirmación.
6. Abrir el archivo Excel desde la carpeta donde fue guardado.

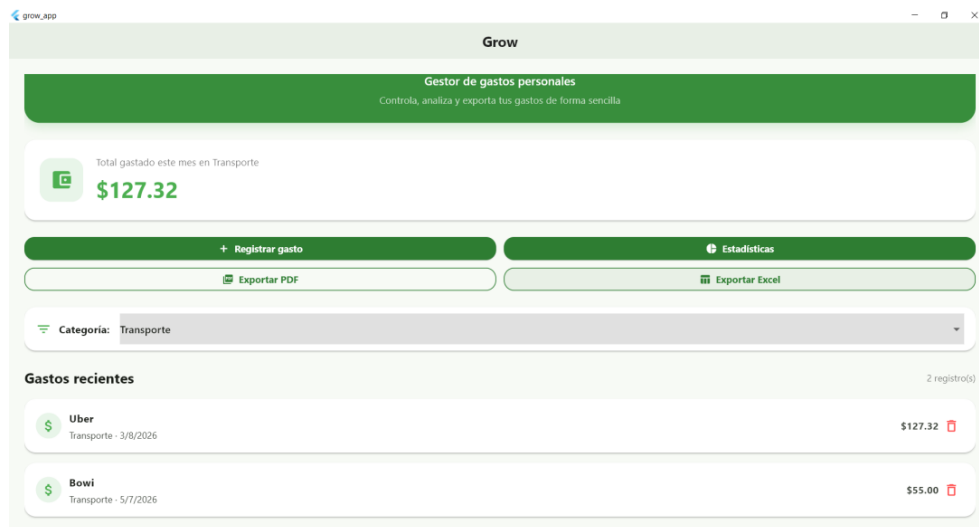


Ilustración 16. Pantalla de botón de exportación de Excel

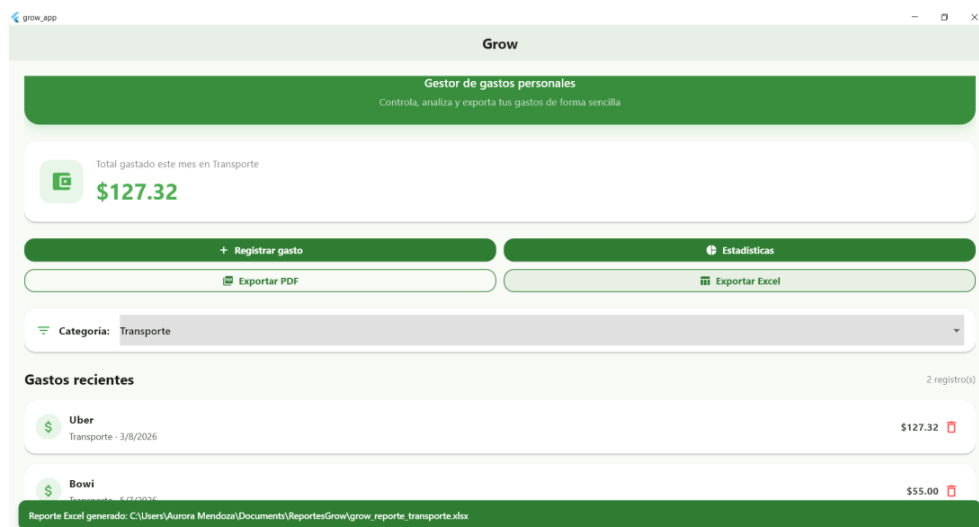


Ilustración 17. Pantalla de mensaje de exportación a Excel

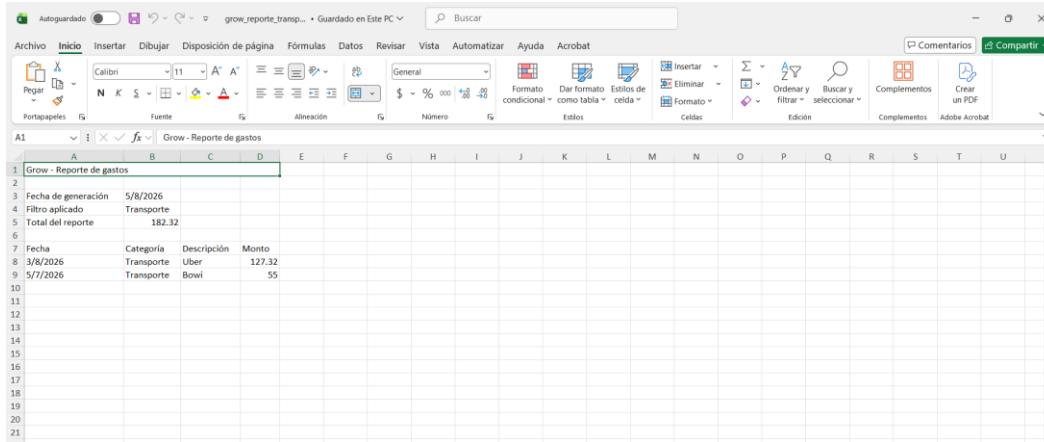


Ilustración 18. Pantalla de archivo Excel exportado

14.10 Sitio web de documentación

Además de la aplicación de escritorio, Grow cuenta con un sitio web de documentación publicado mediante GitHub Pages. Este sitio permite consultar información general del proyecto desde un navegador. Esta página complementa la documentación técnica y permite presentar el proyecto de una manera más visual y accesible.

El sitio incluye secciones como:

- Descripción del proyecto
- Objetivo general
- Funcionalidades principales
- Herramientas utilizadas
- Manual básico de uso
- Estructura del proyecto
- Pruebas de calidad
- Documento técnico
- Estado actual
- Conclusión

El procedimiento para consultar el sitio es el siguiente:

1. Abrir un navegador web.
2. Ingresar al sitio publicado de Grow.
3. Revisar las secciones disponibles.
4. Consultar la información general del proyecto.

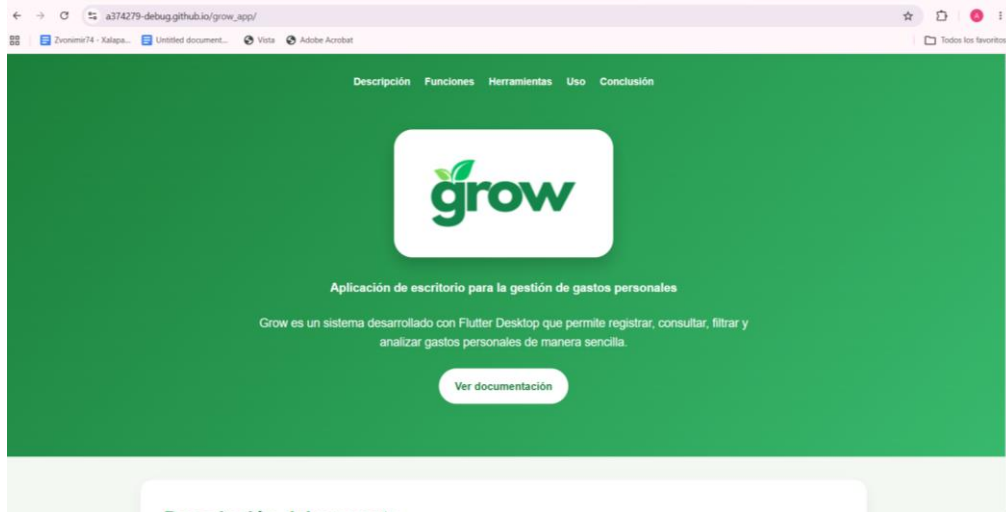


Ilustración 19. Pantalla principal del sitio web de Grow

14.11 Recomendaciones de uso

Para utilizar Grow de manera adecuada, se hacen las siguientes recomendaciones:

- ✓ Registrar los gastos con datos completos.
- ✓ Revisar que el monto ingresado sea correcto.
- ✓ Seleccionar la categoría adecuada para cada gasto.
- ✓ Utilizar descripciones claras.
- ✓ Verificar la fecha antes de guardar.
- ✓ Consultar periódicamente la pantalla de estadísticas.
- ✓ Exportar reportes PDF o Excel como respaldo.
- ✓ No eliminar gastos si todavía se necesitan para el análisis.

15 Estado actual del proyecto y conclusiones personales

Grow se encuentra en una etapa funcional y de documentación final. Las características principales definidas dentro del alcance fueron implementadas y verificadas, por lo cual la aplicación puede usarse para registrar, almacenar, consultar, filtrar, analizar y exportar gastos personales desde un equipo con Windows.

La aplicación cuenta con una interfaz gráfica desarrollada mediante **Flutter Desktop**, mientras que la lógica del sistema se programó en **Dart**. Los gastos se almacenan localmente en una base de datos **SQLite**, lo que permite conservar la información sin depender de una conexión a Internet.

Funcionalidades implementadas

Funcionalidad	Estado
Registro de gastos con monto, categoría, descripción y fecha	Implementada
Validación de los datos del formulario	Implementada
Almacenamiento local mediante SQLite	Implementado
Consulta y visualización de gastos	Implementada
Eliminación de gastos	Implementada
Filtrado de gastos por categoría	Implementado
Cálculo del total mensual	Implementado
Cálculo de total, cantidad y promedio de gastos	Implementado
Agrupación de gastos por categoría	Implementada
Pantalla de estadísticas	Implementada
Representación gráfica de estadísticas	Implementada
Exportación de reportes PDF	Implementada
Exportación de reportes Excel	Implementada
Integración del logo e identidad visual de Grow	Implementada
Control de versiones mediante Git	Implementado
Repositorio remoto en GitHub	Publicado
Sitio de documentación en GitHub Pages	Publicado

Cumplimiento de los objetivos

Objetivo	Nivel de cumplimiento
Desarrollar una aplicación de escritorio para gastos personales	Cumplido
Registrar y almacenar gastos	Cumplido
Consultar y filtrar información	Cumplido
Generar estadísticas	Cumplido
Exportar reportes PDF y Excel	Cumplido
Aplicar pruebas unitarias y de widgets	Cumplido
Evaluar el rendimiento	Cumplido
Analizar posibles fallos	Cumplido
Controlar versiones con Git y GitHub	Cumplido

Objetivo	Nivel de cumplimiento
Publicar documentación web	Cumplido
Elaborar documentación técnica	Cumplido

Estado de los componentes principales

Componente o actividad	Estado actual	Observaciones
Registro de gastos	Completado	El formulario permite capturar y validar la información
Base de datos SQLite	Completado	Los gastos se almacenan y recuperan localmente
Lista de gastos	Completado	Los registros aparecen en la pantalla principal
Eliminación de gastos	Completado	El usuario puede eliminar registros almacenados
Filtro por categoría	Completado	Permite mostrar gastos de una categoría específica
Total mensual	Completado	Se calcula con base en los gastos del mes actual
Estadísticas	Completado	Incluye total, cantidad, promedio y agrupación por categoría
Gráficos	Completado	Representan visualmente la distribución de los gastos
Exportación PDF	Completado	El reporte se genera y se guarda correctamente
Exportación Excel	Completado	La información se organiza en una hoja de cálculo
Pruebas automatizadas	Completado (para el alcance definido)	Todas las pruebas implementadas fueron aprobadas
Evaluación de rendimiento	Completado	Se realizó una prueba general mediante Flutter DevTools
Árboles de fallos	Completado	Se analizaron tres eventos superiores
Control de versiones	Completado	El proyecto se encuentra respaldado en GitHub
Sitio en GitHub Pages	Completado	La documentación básica se encuentra publicada
Documento técnico	Completado	Se están integrando las últimas secciones y evidencias
Manual de usuario	Completado	Debe integrarse como parte de la entrega final.

Cumplimiento de los objetivos

Objetivo	Nivel de cumplimiento
Desarrollar una aplicación de escritorio para gastos personales	Cumplido
Registrar y almacenar gastos	Cumplido
Consultar y filtrar información	Cumplido
Generar estadísticas	Cumplido
Exportar reportes PDF y Excel	Cumplido
Aplicar pruebas unitarias y de widgets	Cumplido
Evaluar el rendimiento	Cumplido
Analizar posibles fallos	Cumplido
Controlar versiones con Git y GitHub	Cumplido
Publicar documentación web	Cumplido
Elaborar documentación técnica	Cumplido

Limitaciones actuales

- Está orientada principalmente a equipos con Windows.
- Los datos se almacenan únicamente en el equipo local.
- No existe sincronización ni respaldo automático en la nube.
- No dispone de autenticación o gestión de diferentes usuarios.
- No se conecta con cuentas bancarias o servicios financieros.
- Las pruebas automatizadas cubren componentes seleccionados, pero no todos los posibles escenarios.
- La evaluación de rendimiento fue general y no una prueba de carga prolongada.
- Los árboles de fallos no incluyen probabilidades cuantitativas.
- No se ha desarrollado una versión móvil o web funcional de la aplicación.
- No cuenta con autenticación ni administración de múltiples usuarios.

Conclusiones personales

Desde una perspectiva meramente académica, la realización de este mi primer proyecto de desarrollo de software representó un desafío muy complejo. Hasta el momento, las actividades desarrolladas durante la carrera fueron de naturaleza más bien teórica, de modo que aterrizar conceptos tan abstractos y darles una cara real fue una mezcla muy peculiar entre incertidumbre y satisfacción por concluir (al menos esta primera versión) la aplicación que solamente había teorizado en cuatrimestres anteriores.

En la consciencia de que mi aplicación **Grow** tiene mucho que mejorar aún, lo considero un proyecto que me ha abierto el camino a continuar mi preparación en la ingeniería de software.

16 Anexo gráfico

16.1 Diagrama UML de clases

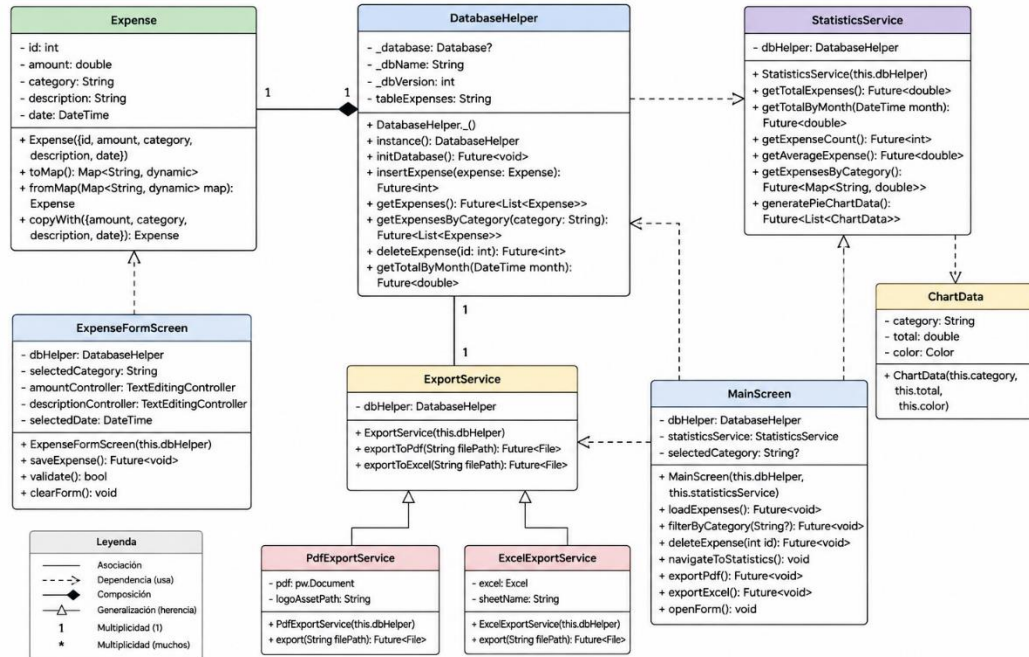


Ilustración 20. Diagrama UML de clases

16.2 Diagrama ER

Caso de uso “Registro de gasto”

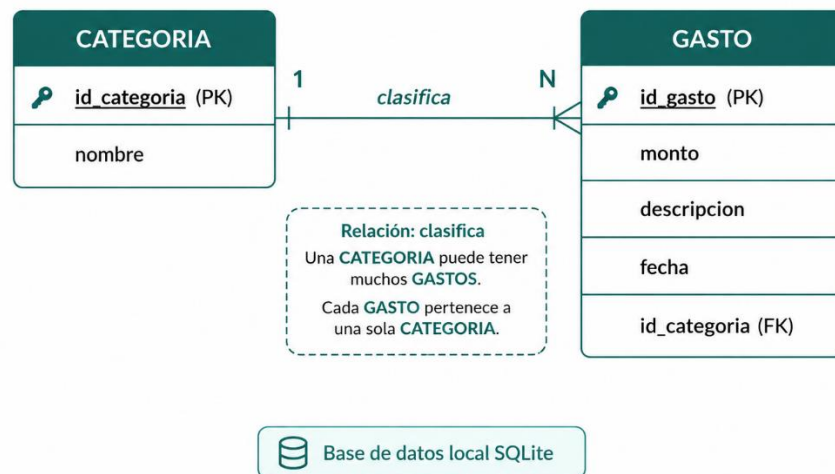


Ilustración 21. Diagrama Entidad-Relación de Grow

16.3 Interfaces

Pantalla principal

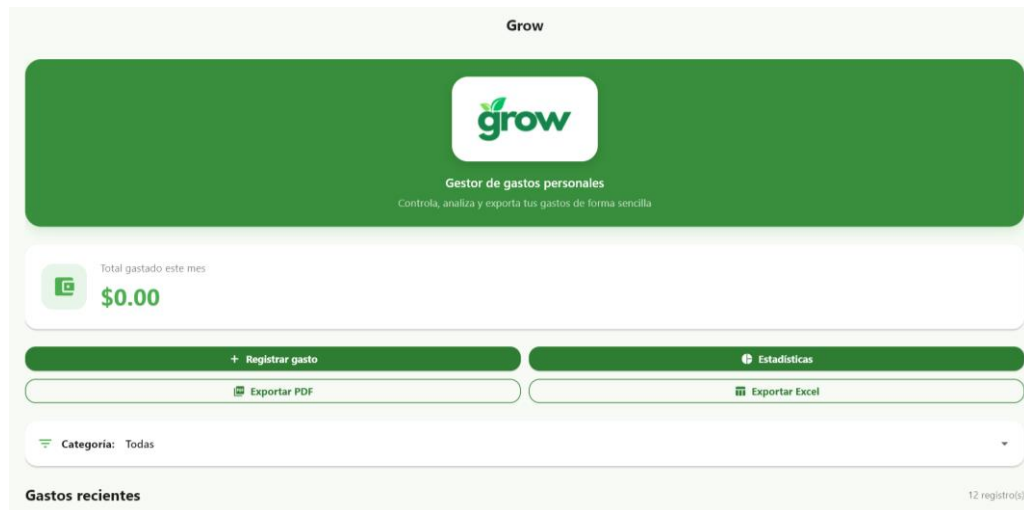


Ilustración 6. Pantalla principal de Grow

Pantallas de registro de gastos

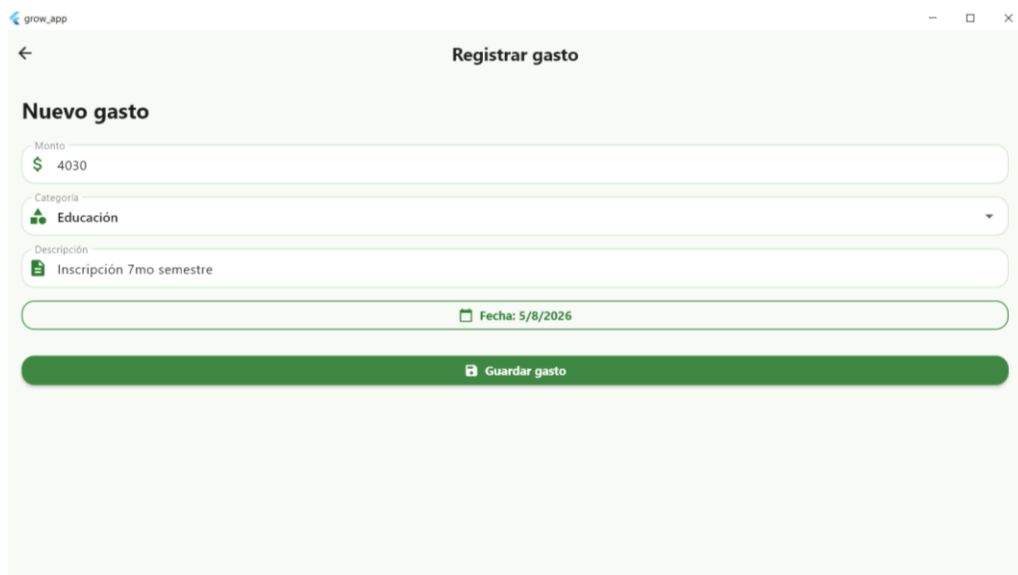


Ilustración 7. Pantalla de registro de gastos

Pantalla de consulta de gastos registrados

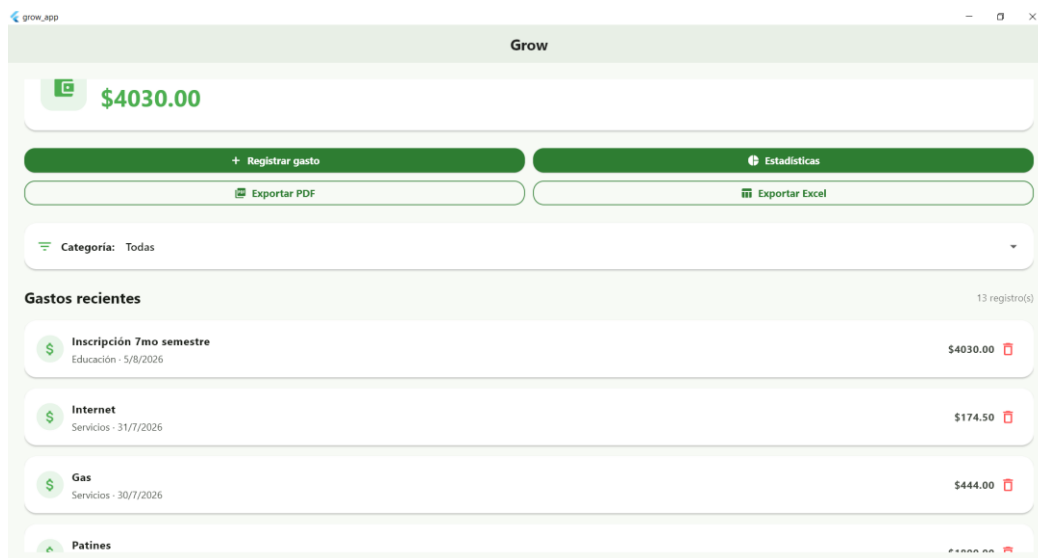


Ilustración 8. Pantalla de lista de gastos registrados

Pantallas de filtrado por categoría

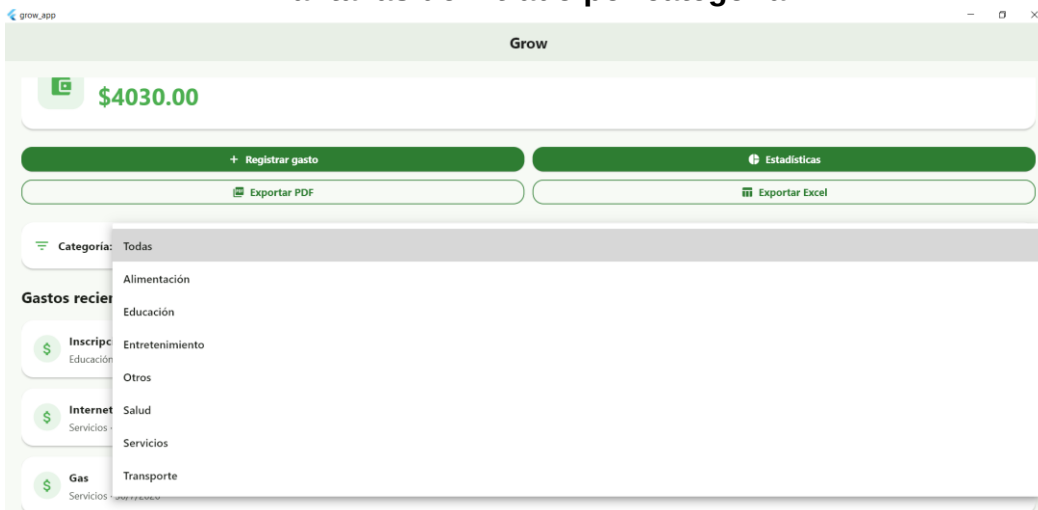


Ilustración 9. Pantalla filtro de gastos por categoría

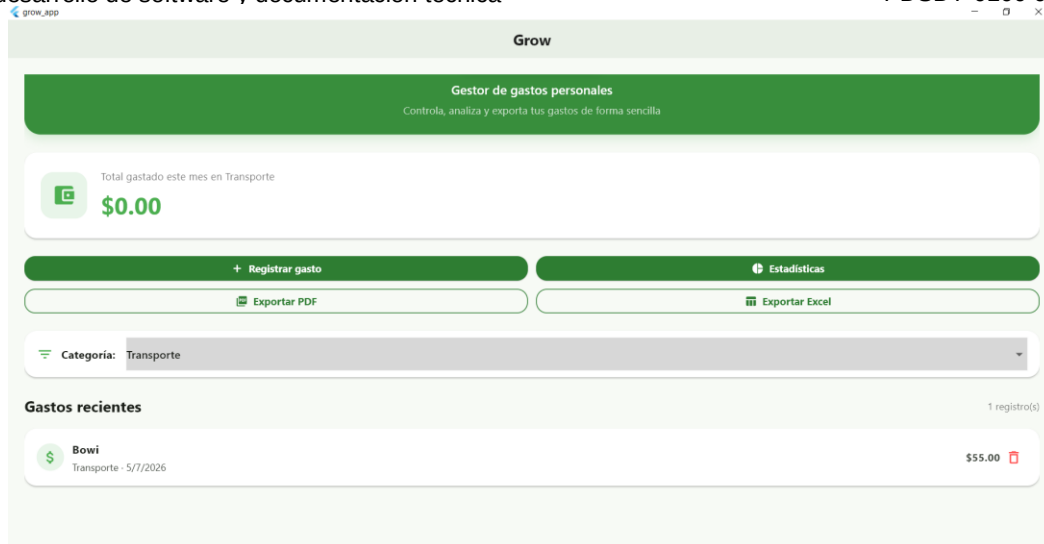


Ilustración 10. Pantalla de lista de gastos filtrados por categoría “Transporte”

Pantalla de eliminación de gasto

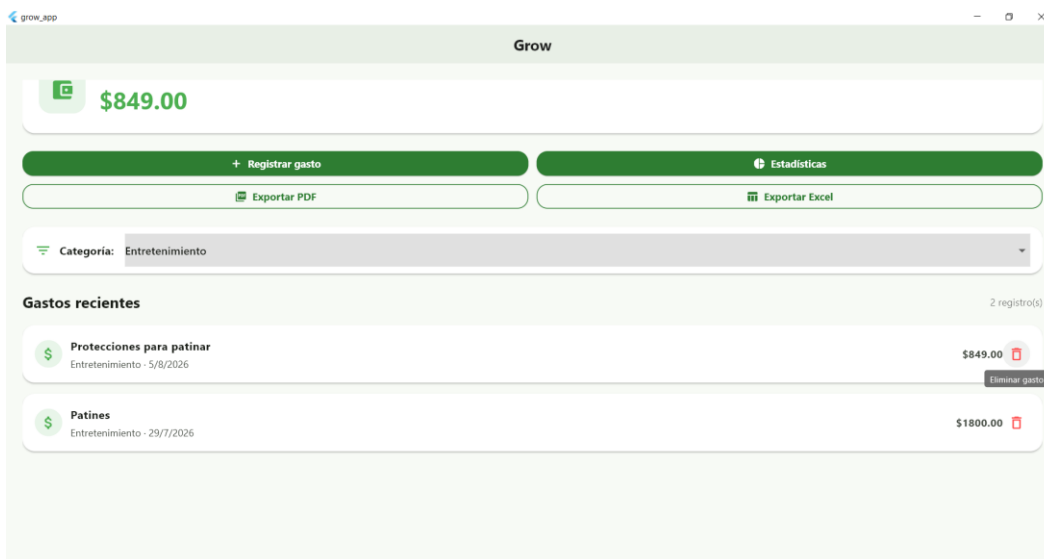


Ilustración 11. Pantalla de botón de eliminación de gasto

Pantalla de estadísticas

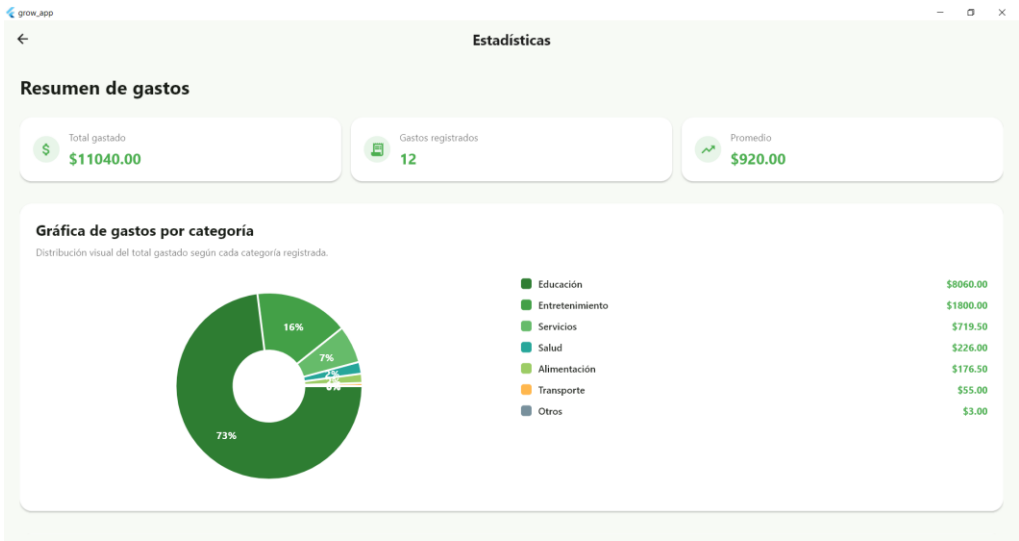


Ilustración 12. Pantalla de gráfico de gastos

Pantallas de exportación de reportes

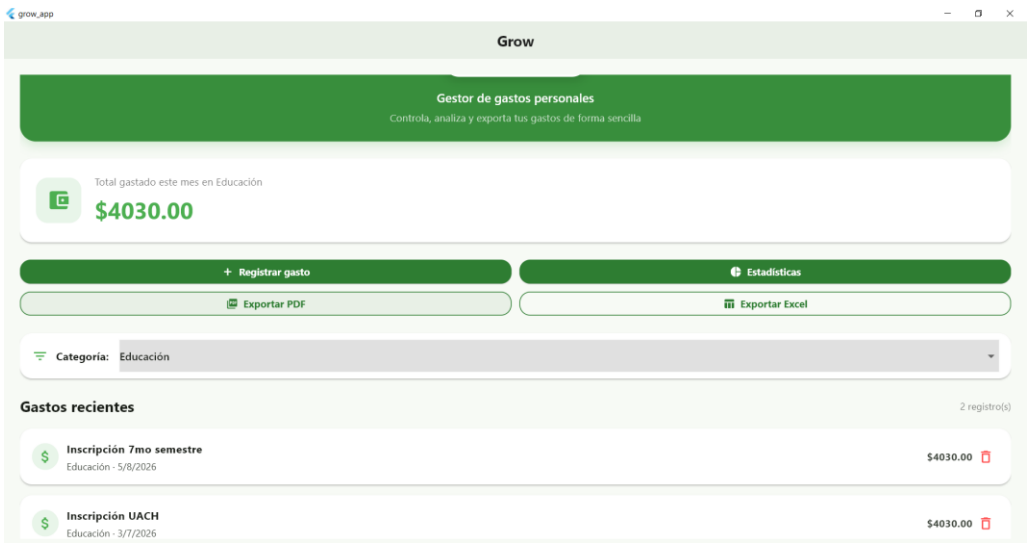


Ilustración 13. Pantalla de botón “Exportar PDF”

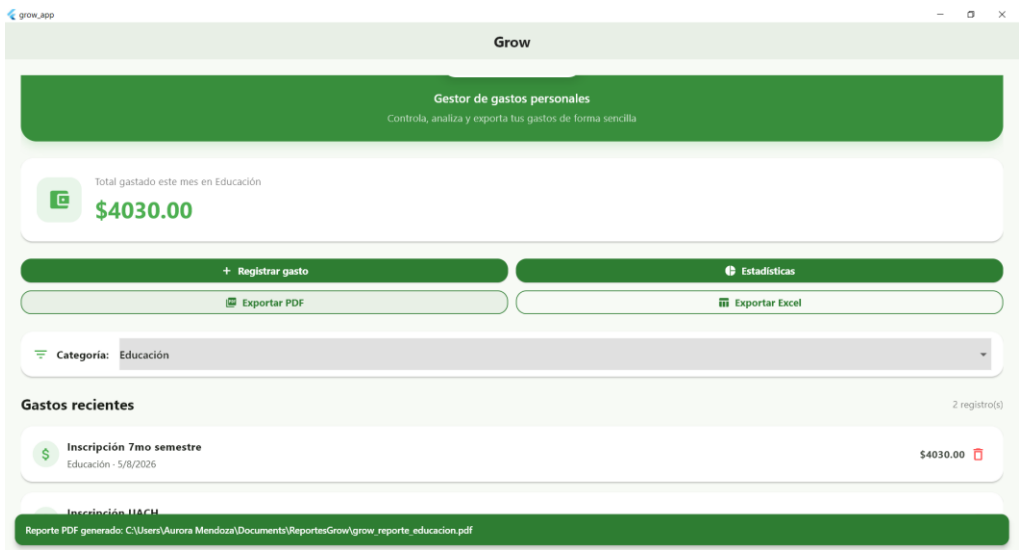


Ilustración 14. Pantalla de mensaje de exportación de PDF

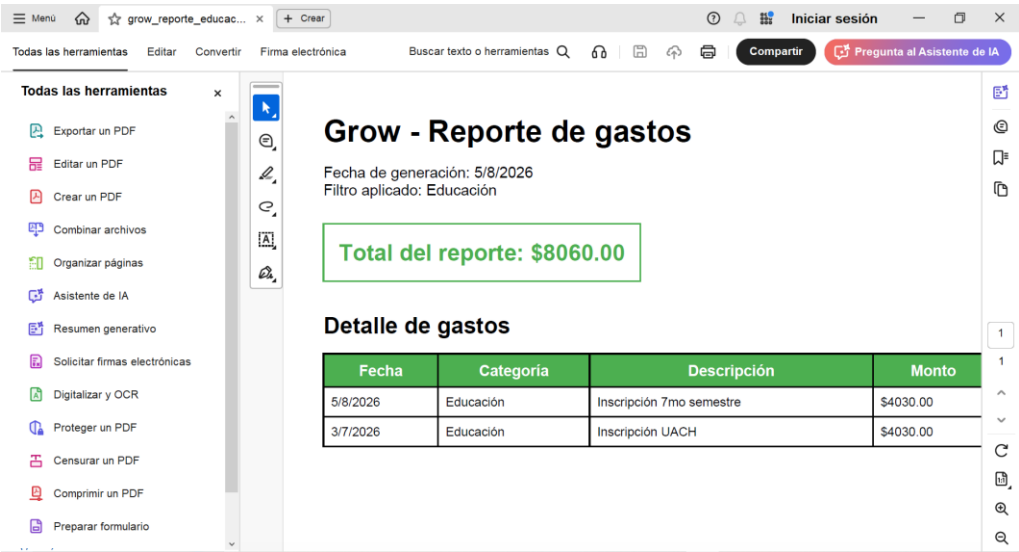


Ilustración 15. Pantalla del archivo PDF exportado

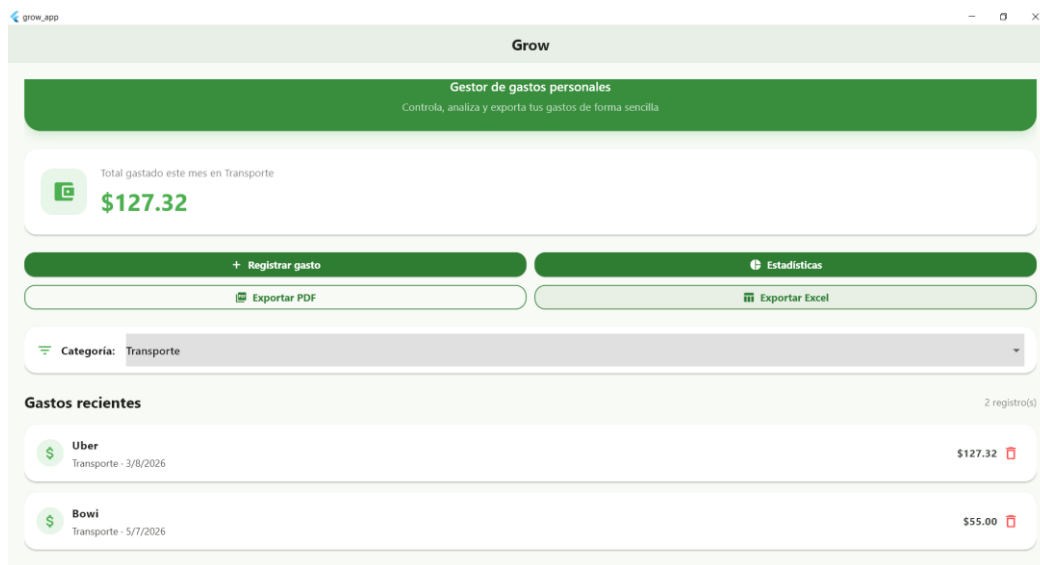


Ilustración 16. Pantalla de botón de exportación de Excel

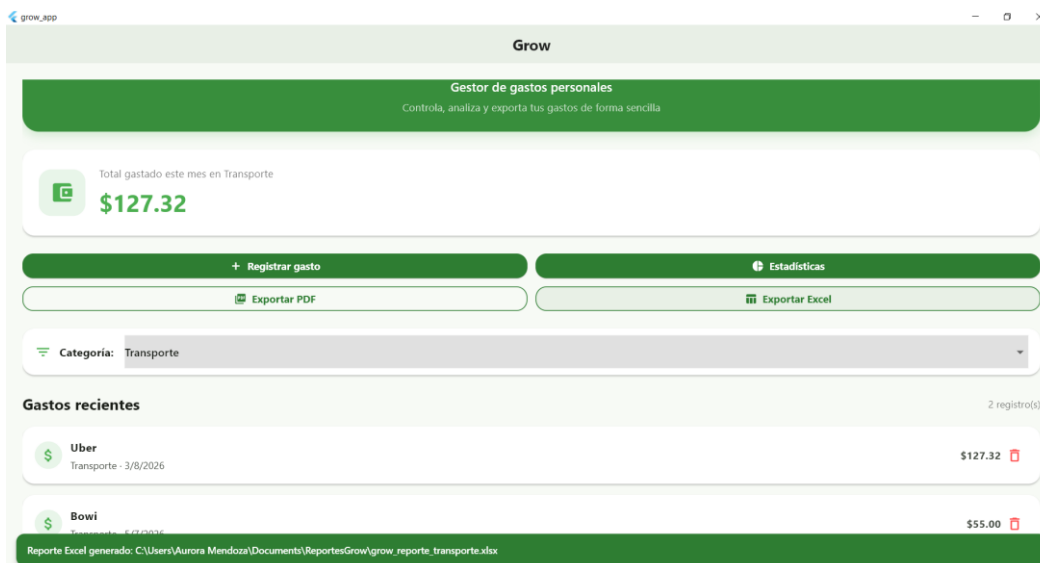
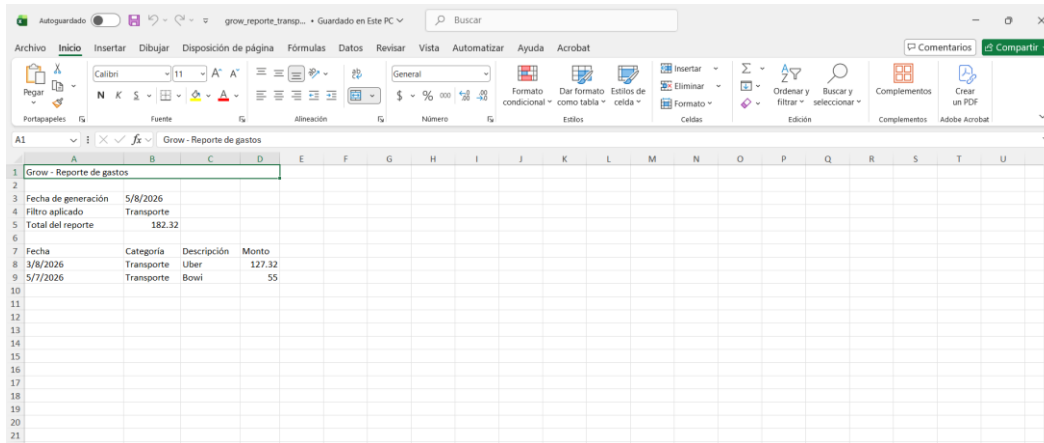


Ilustración 17. Pantalla de mensaje de exportación a Excel



The screenshot shows an Excel spreadsheet with the following data:

Grow - Reporte de gastos			
Fecha de generación	5/8/2026		
Filtro aplicado	Transporte		
Total del reporte	182.32		
Fecha	Categoría	Descripción	Monto
3/8/2026	Transporte	Uber	127.32
5/7/2026	Transporte	Bowli	55

Ilustración 18. Pantalla de archivo Excel exportado

Sitio web de GitHub Pages

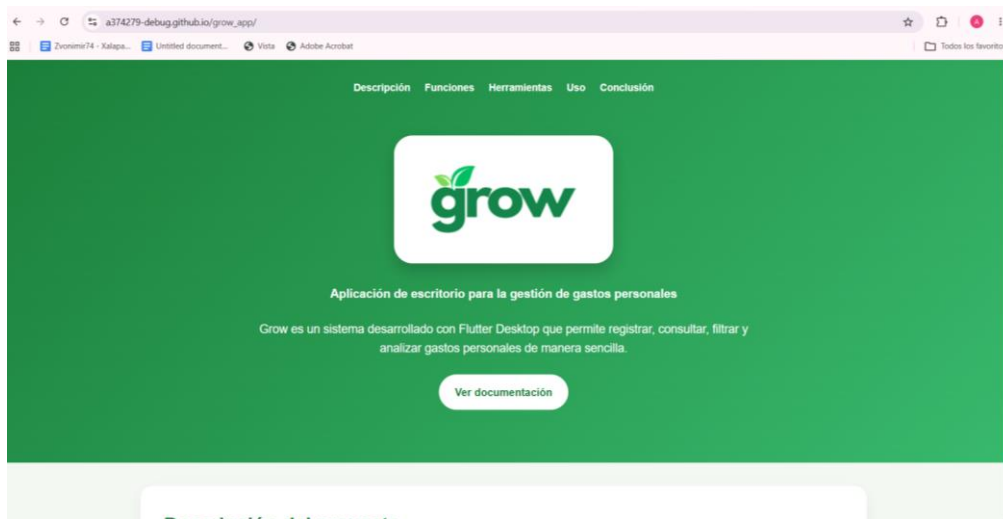
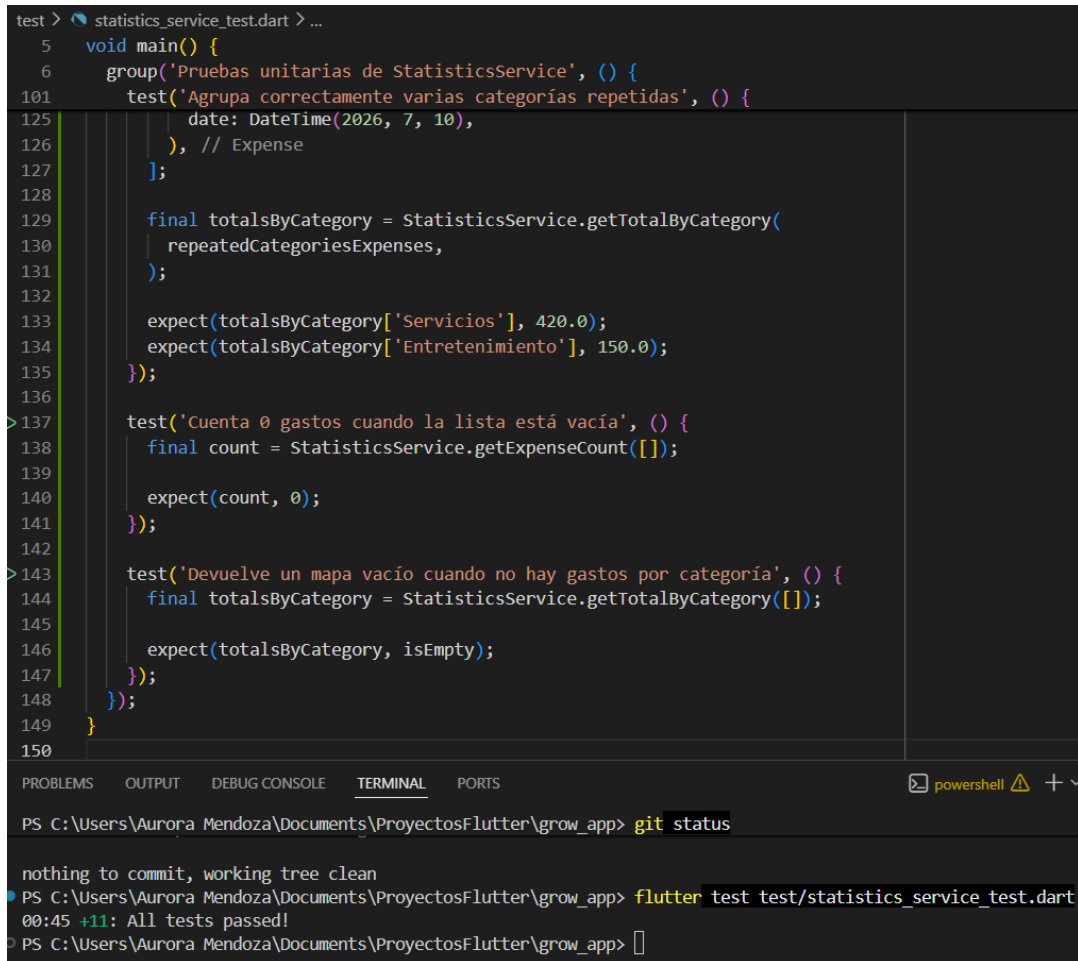


Ilustración 19. Pantalla principal del sitio web de Grow

16.4 Pantallas de pruebas de software con Flutter Test

Pruebas unitarias



```
test > statistics_service_test.dart > ...
5  void main() {
6    group('Pruebas unitarias de StatisticsService', () {
101   test('Agrupa correctamente varias categorías repetidas', () {
125     date: DateTime(2026, 7, 10),
126   }, // Expense
127   );
128
129   final totalsByCategory = StatisticsService.getTotalByCategory(
130     repeatedCategoriesExpenses,
131   );
132
133   expect(totalsByCategory['Servicios'], 420.0);
134   expect(totalsByCategory['Entretenimiento'], 150.0);
135   });
136
137   test('Cuenta 0 gastos cuando la lista está vacía', () {
138     final count = StatisticsService.getExpenseCount([]);
139
140     expect(count, 0);
141   });
142
143   test('Devuelve un mapa vacío cuando no hay gastos por categoría', () {
144     final totalsByCategory = StatisticsService.getTotalByCategory([]);
145
146     expect(totalsByCategory, isEmpty);
147   });
148   });
149 }
150
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS powershell +

```
PS C:\Users\Aurora Mendoza\Documents\ProyectosFlutter\grow_app> git status
nothing to commit, working tree clean
PS C:\Users\Aurora Mendoza\Documents\ProyectosFlutter\grow_app> flutter test test/statistics_service_test.dart
00:45 +11: All tests passed!
PS C:\Users\Aurora Mendoza\Documents\ProyectosFlutter\grow_app>
```

Ilustración 22. Evidencia de pruebas unitarias exitosas

Pruebas de widgets

```
test > add_expense_screen_test.dart > ...
5  void main() {
6  group('Pruebas de widgets de AddExpenseScreen', () {
22    testWidgets(
23      await tester.pumpWidget(const MaterialApp(home: AddExpenseScreen()));
26
27      await tester.tap(find.text('Guardar gasto'));
28      await tester.pumpAndSettle();
29
30      expect(find.text('Ingresa el monto del gasto'), findsOneWidget);
31      expect(find.text('Ingresa una descripción'), findsOneWidget);
32    },
33  );
34
35  testWidgets('Muestra error cuando el monto no es válido', (
36    WidgetTester tester,
37  ) async {
38    await tester.pumpWidget(const MaterialApp(home: AddExpenseScreen()));
39
40    await tester.enterText(find.byType(TextFormField).first, 'abc');
41    await tester.enterText(find.byType(TextFormField).last, 'Comida');
42
43    await tester.tap(find.text('Guardar gasto'));
44    await tester.pumpAndSettle();
45
46    expect(find.text('Ingresa un monto válido'), findsOneWidget);
47  });
48  });
49  }
50  |
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS powershell ⚠ + v

```
PS C:\Users\Aurora Mendoza\Documents\ProyectosFlutter\grow_app> git status
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
● PS C:\Users\Aurora Mendoza\Documents\ProyectosFlutter\grow_app> flutter test test/add_expense_screen_test.dart
00:57 +3: All tests passed!
○ PS C:\Users\Aurora Mendoza\Documents\ProyectosFlutter\grow_app> |
```

Ilustración 23. Evidencia de pruebas de widgets exitosas

16.5 Pantallas de rendimiento con Flutter DevTools



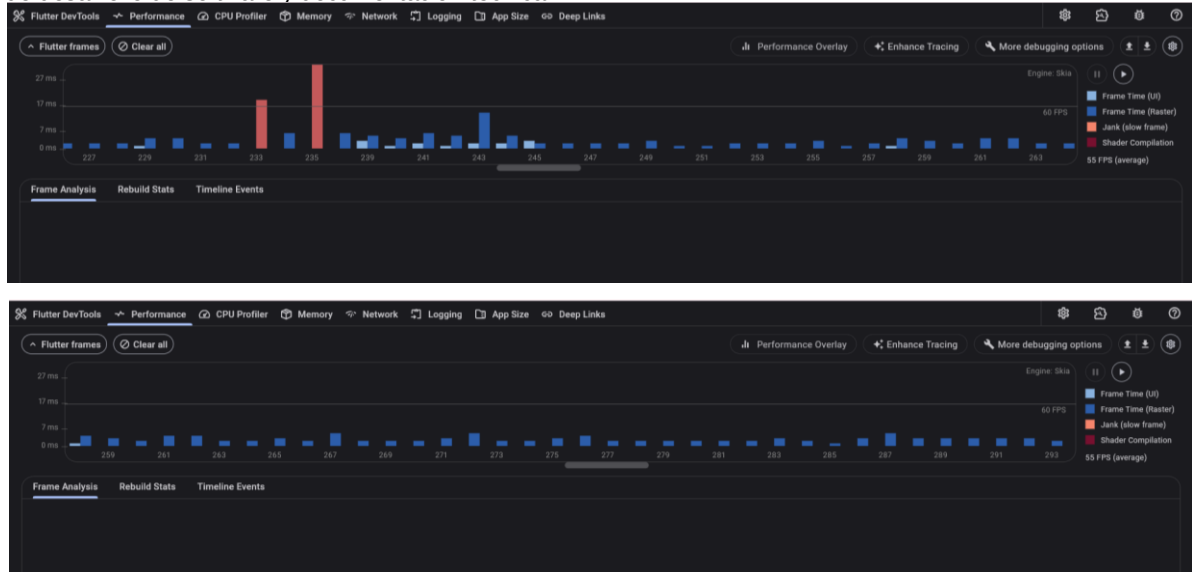


Ilustración 24. Evidencia de prueba de rendimiento aceptable

16.6 Árboles de fallos con ALD

- **Error: “El gasto válido no se registra o no aparece en la lista”**

Creación del evento principal del árbol de fallos

Fault Tree properties

Code: GROW-FTA-01

Description: **Árbol de fallos para analizar el evento: El gasto válido no se registra o no aparece en la lista.**

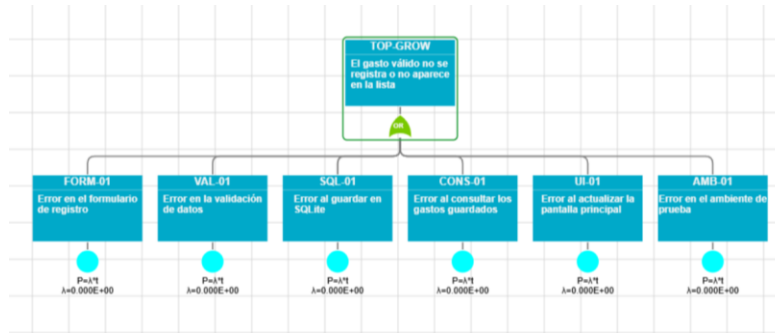
Calc. Types: Unavailability at time t $Q(t)$

Mission time t : 0 (hours)

Close Save changes



Las 6 causas principales debajo de la compuerta OR



Las causas secundarias (posibles causas para cada tipo de error)

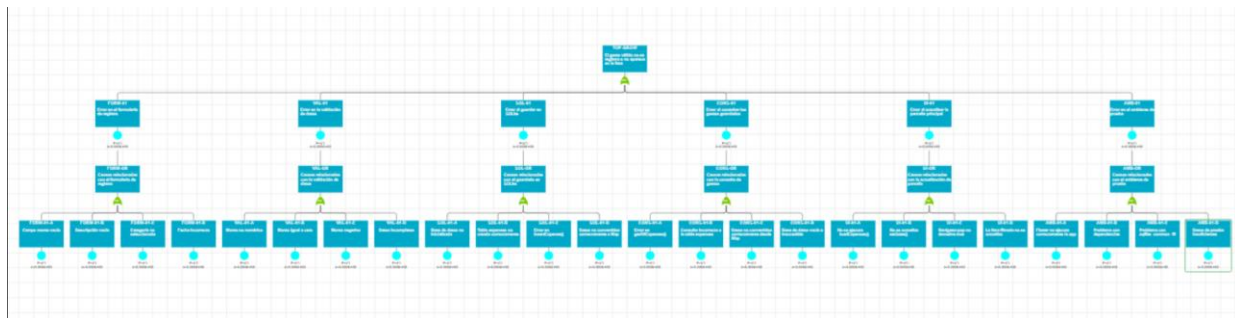


Ilustración 25. Evidencia 1 de creación de árboles de fallos en ALD

- **Error: “El reporte PDF no se genera correctamente”**

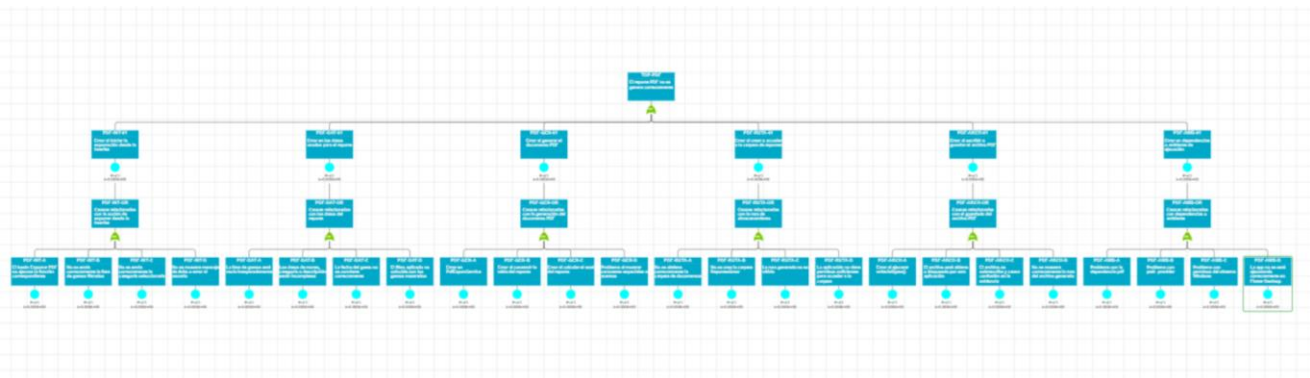


Ilustración 26. Evidencia 2 de árbol de fallos en ALD

- **Error: “La aplicación se cierra inesperadamente”**

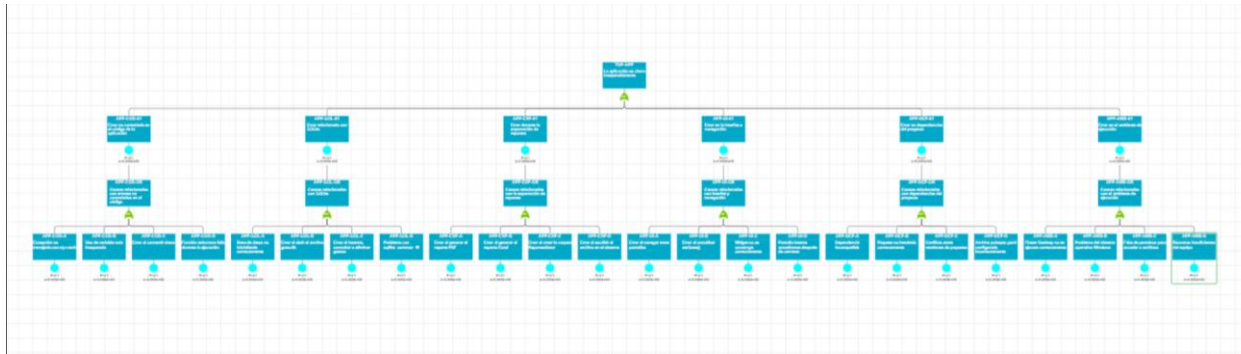


Ilustración 27. Evidencia 3 de árbol de fallos en ALD

17 Bibliografía

ALD Reliability, Safety And Quality Blog. (28 de abril, 2014). *Free Fault Tree analysis software*. <https://aldreliability.wordpress.com/2014/04/28/free-fault-tree-analysis-software/>

Apache JMeter. (s. f.). *Apache JMeterTM*. <https://jmeter.apache.org/>

Atlassian. (s. f.). *Qué es Trello: descubre sus funciones, usos y todo lo que ofrece*. <https://trello.com/es/tour>

Etter, Andrew (2016). *Modern Technical Writing: An Introduction to Software Documentation*. Recuperado el 17 de julio, 2026. <https://pdfcoffee.com/modern-technical-writing-an-introduction-to-software-documentation-by-andrew-etter-pdf-free.html>

Flutter Docs. (s. f.). *Testing & debugging*. <https://docs.flutter.dev/testing>

GitBook. (s. f.). *Do I have to pay to publish a site? | Help Center | GitBook Documentation*. <https://gitbook.com/docs/help-center/account-management/pricing-plans-and-billing/do-i-have-to-pay-to-publish-a-site>

GitHub Docs. (s. f.). *Acerca de las wikis - Documentación de GitHub*. <https://docs.github.com/es/communities/documenting-your-project-with-wikis/about-wikis>

GitHub Docs. (s. f.). *Continuous integration*. <https://docs.github.com/en/actions/get-started/continuous-integration>

GitHub Docs. (s. f.). *What is GitHub Pages? - GitHub Docs*. <https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages>

Gómez Fuentes, M. del C. (2011). *Notas del curso: Análisis de requerimientos*. Departamento de Matemáticas Aplicadas y Sistemas, Universidad Autónoma Metropolitana. ISBN 978-607-477-442-9.

SL, D. S. (2004). *Sobre las Pruebas de Usuario - Inicio [ergoestudio.com] Análisis Visual y Test de Usabilidad*. <http://www.ergoestudio.com/inicio/preguntas/pruebas.php>

University of West Florida. Institute for Human and Machine Cognition. (2000). *IHMC CmapTools*, Institute for Human and Machine Cognition. Recuperado en el 17 de julio de 2026, del sitio Web temoa: Portal de Recursos Educativos Abiertos (REA). <http://www.temoa.info/es/node/173>

Universidad, U. (2025, 31 de marzo). *Trello: Herramienta Eficaz para la Gestión de Proyectos*. Universidad Americana de Europa. <https://unade.edu.mx/trello-herramienta-eficaz-para-la-gestion-de-proyecto>

